14st-10-2026

International Cybersecurity Of Digital Forensic Academy (ICDFA)

PROGRAMME: Advance Critical Infrastructure Secuity (ACIS)

Prepared by: Aminu Ibrahim Jalo
Course: ACICP302-ICS Network Security & Attacks
(Module 1 Integrated ICS Network Security & Attacks: Analyze, Test & Recover the Supplied OT Simulation)

Tittle: Analyze, Test & Defend the Supplied Modbus/TCP OT Simulation

Assignment: Week 1-Labs Report
C11/26/ACIS/17359

Instructor : Aminu Idris

Student Name	Aminu Ibrahim Jalo
Registration Number	C11/26/ACIS/17359
Signature	
Date	14st-10-2026

Scenario

You are an OT security analysis team assigned a small simulated process based on the course-provided Modbus/TCP source. Your task is to inspect and run the supplied loopback-safe simulation in an isolated Ubuntu virtual machine, establish normal operation, analyze protocol and guard behavior, complete controlled demonstrations, assess limitations and verify recovery. You are not required to build a replacement simulator or guard.

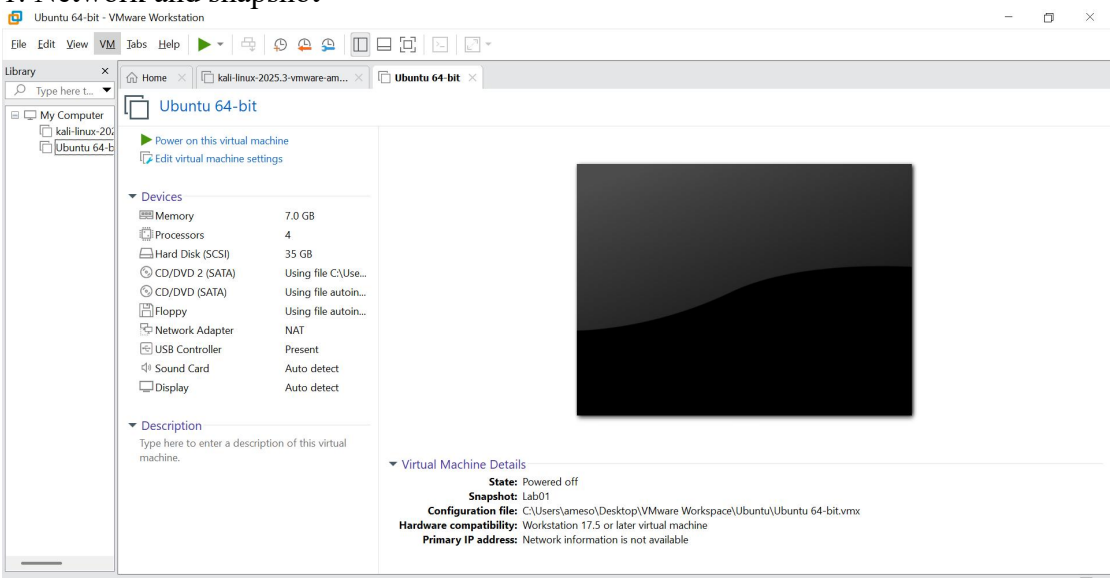
Objectives

- Safely locate and verify the canonical course source, record its commit and run it only inside the assigned Ubuntu VM on loopback.
- Explain the supplied architecture, process model, Modbus/TCP data points and direct/guarded paths using source and runtime evidence.
- Capture normal loopback traffic, identify a request/response and connect packet fields to process state.
- Document the local service exposure and process-related risks within the authorized scope.
- Analyze the supplied guard's checks, events, assumptions and limitations without misrepresenting capabilities.
- Plan and run at least three approved bounded demonstrations, compare expected and observed results, and verify recovery.
- Submit a private evidence repository with traceable, learner-created artifacts and defend conclusions accurately.

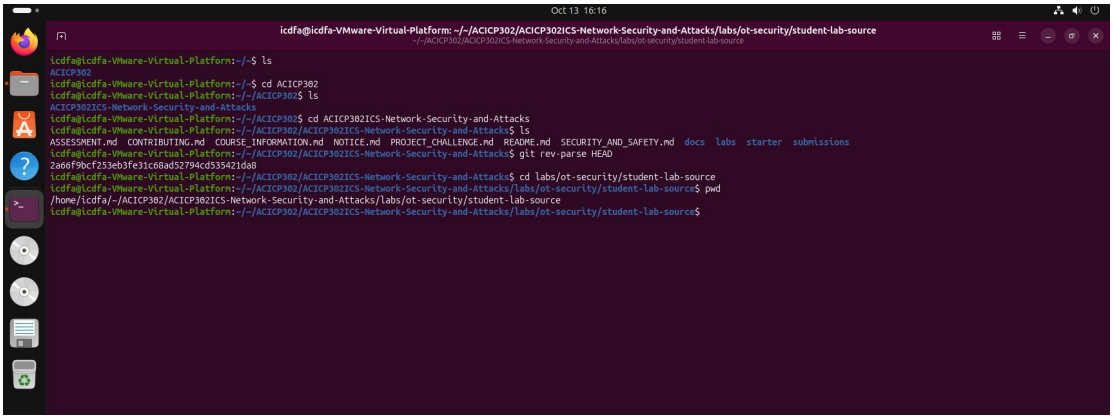
Activity 00 - Lab preflight and safety

Began with prepared the assigned Ubuntu VM and verified the exact course source and authorization boundary before starting any simulation process.

1. Network and snapshot



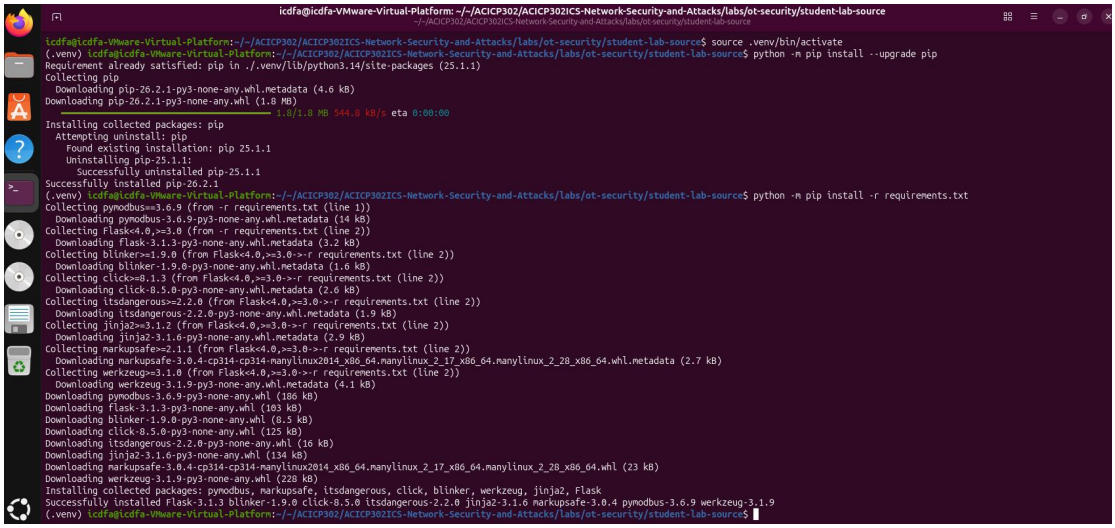
1. Source and commit



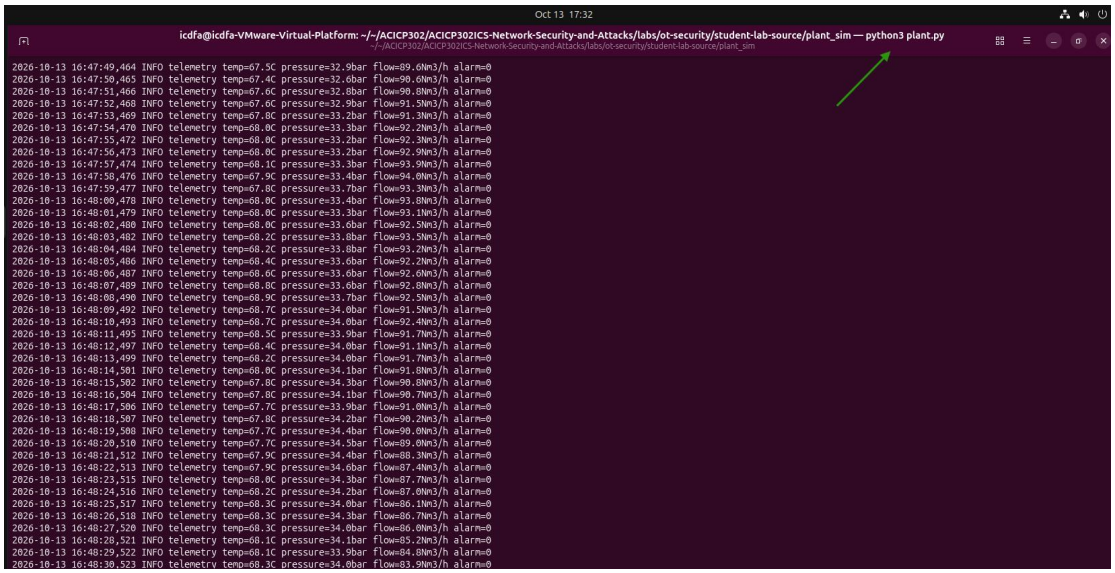
2. Loopback check

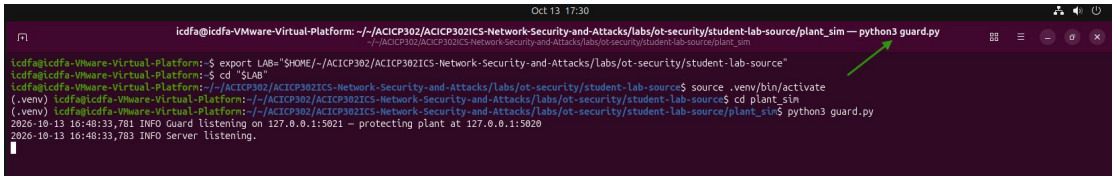


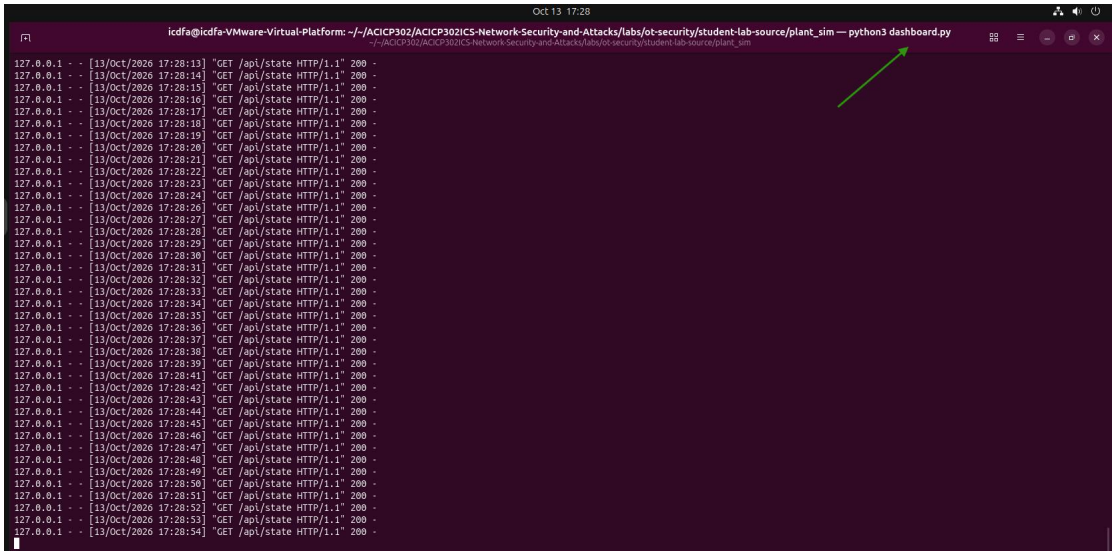
3. Environment

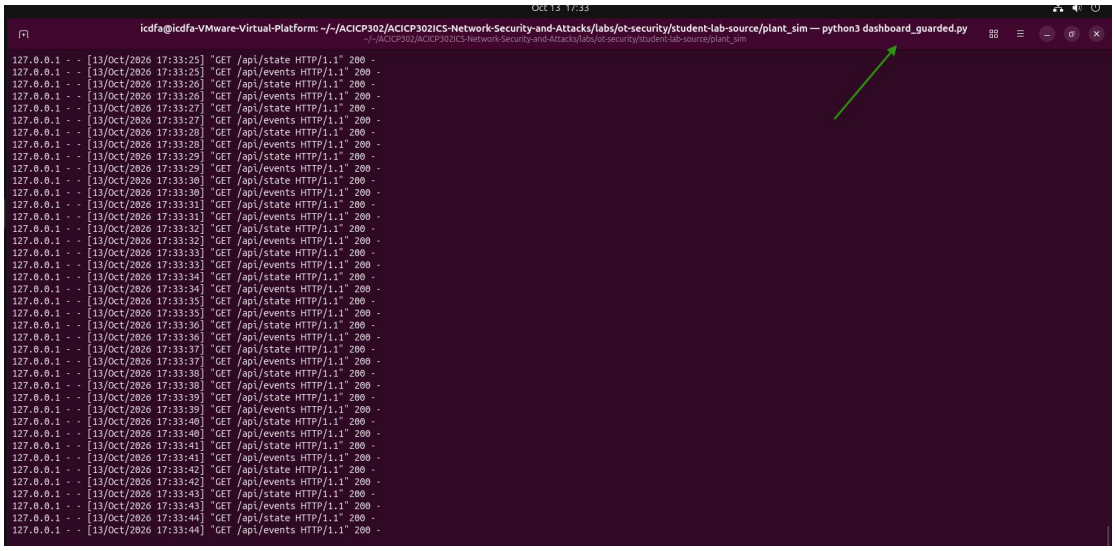


All the 4 terminals are in Venv









Activity 01 - Architecture and plant baseline

1. Objective

The objective of Activity 01 was to establish and document a baseline for the authorized industrial-control-system training environment. This included identifying the virtual machine, recording the date and time, checking the locally exposed services, observing the dashboard under undefended and guarded conditions, and preserving evidence for subsequent activities.

2. Environment Verification

The activity was performed inside the assigned Ubuntu virtual machine.



Hostname: icdfa-VMware-Virtual-Platform

Date: 13 October 2026

Time zone: Africa/Lagos (WAT)

Evidence directory: ~/ACICP302

Network scope: Local loopback services at 127.0.0.1

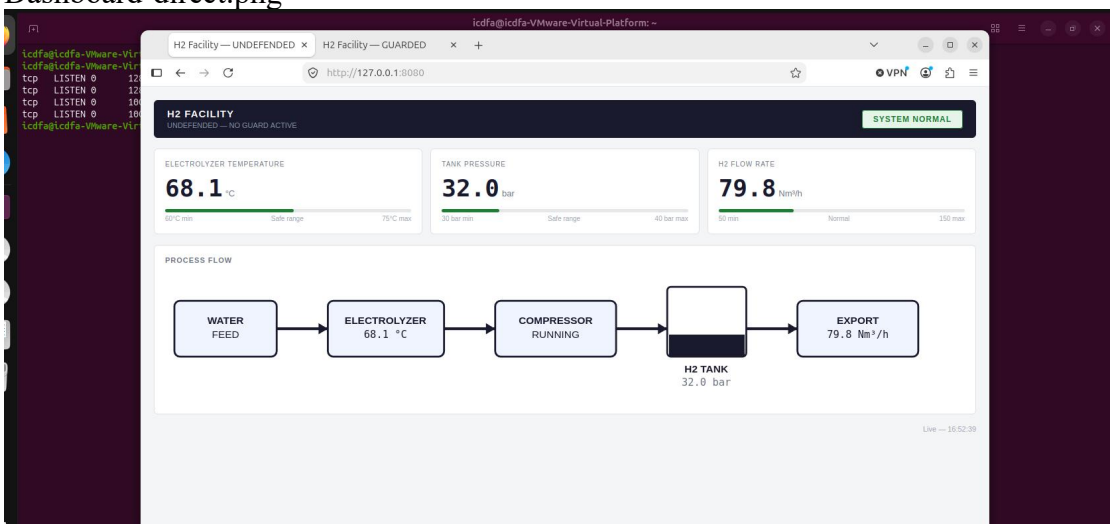
Source commit: To be recorded from the authorized course repository.
Network adapter isolation: To be confirmed and documented from the VM settings.
The local listener check identified four TCP services:

Port	Listening address	Process
5020	127.0.0.1	Python 3
5021	127.0.0.1	Python 3
8080	127.0.0.1	Python 3
8081	127.0.0.1	Python 3

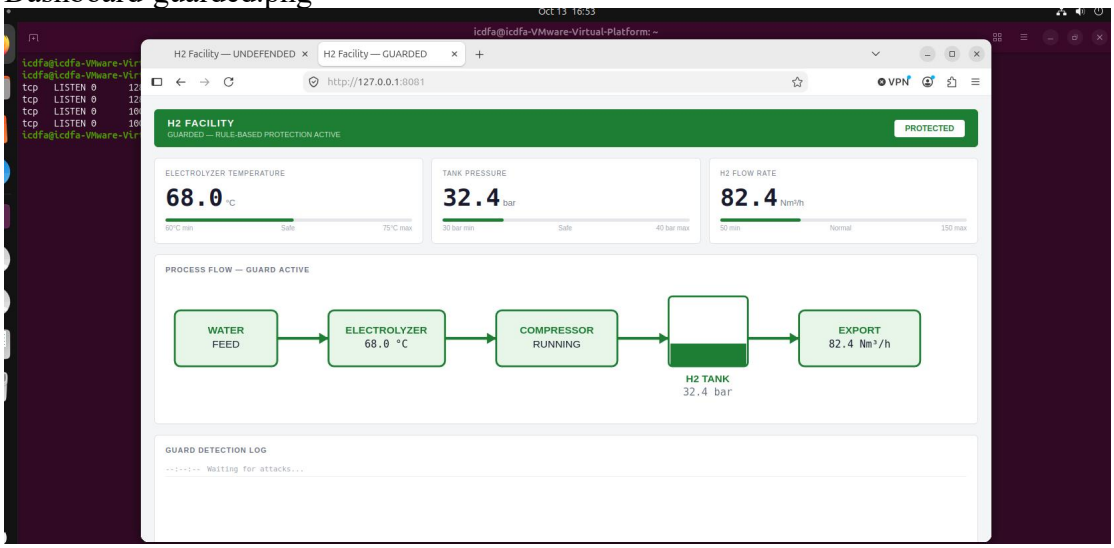
All four services were listening on the loopback interface at the time of observation. This indicates local availability; it does not, by itself, establish application correctness or security.

4. Baseline and packet capture

Dashboard-direct.png



Dashboard-guarded.png



1. Observed readings

Parameter	Direct dashboard (UNDEFENDED)	Guarded dashboard (GUARDED)	Displayed range
Electrolyzer	66.5 °C	67.2 °C	60-75 °C

temperature			
Tank pressure	30.0 bar	32.8 bar	30-40 bar
H_2 flow rate	82.5 Nm^3/h	79.8 Nm^3/h	50-150 Nm^3/h

2. Observation

Baseline Dashboard Observations

Two dashboard states were observed and recorded: an undefended state and a guarded state.

Undefended state: The dashboard was labelled “UNDEFENDED - NO GUARD ACTIVE.” Temperature, pressure and flow rate were within their displayed operating ranges.

Guarded state: The dashboard was labelled “GUARDED - RULE-BASED PROTECTION ACTIVE.” The displayed temperature, pressure and flow rate were also within their respective ranges.

The readings show small differences between the two observed states. These differences are baseline observations only and do not establish that the guard caused the changes or that any attack was blocked.

Conclusion

Activity 01 established an initial record of the virtual machine, locally listening services and the two dashboard states. The observed values provide a reference for later comparisons. Before submission, the source commit, loopback check, adapter isolation evidence and contextual dashboard screenshots should be added to complete the baseline record.

Activity 02 - Modbus/TCP data model and normal operations

Objective

This activity is to understand and document the data model implemented by the supplied industrial control system (ICS) simulation by inspecting its source code and identifying process variables, Modbus register addresses, measurement units, scaling factors, normal operating bounds, alarm/status semantics, and update behavior. The activity also aims to verify selected register readings through the authorized local simulation, correlate the observed values with the dashboard, and explain the Modbus/TCP client-server communication process through a normal request-and-response transaction. Any discrepancies between source-code definitions, comments, displayed values, and observed behavior will be documented, while unknown or unverified details will remain explicitly identified as unknown.

Plant data model

```
icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba plant.py | sed -n '1,100p'
     1  import logging
     2  import random
     3  import threading
     4  import time
     5
     6  from pymodbus.datastore import (
     7      ModbusSequentialDataBlock,
     8      ModbusServerContext,
     9      ModbusSlaveContext,
    10  )
    11  from pymodbus.server import StartTcpServer
    12
    13  logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
    14  log = logging.getLogger("plant")
    15
    16  REG_TEMP = 0
    17  REG_PRESSURE = 1
    18  REG_FLOW = 2
    19  REG_ALARM = 3
    20
    21  TEMP_RANGE = (600, 750)
    22  PRESSURE_RANGE = (300, 400)
    23  FLOW_RANGE = (500, 1500)
    24
    25  UPDATE_INTERVAL_SEC = 1.0
    26  HOST = "127.0.0.1"
    27  PORT = 5020
    28
    29
    30  def clamp(value, low, high):
    31      return max(low, min(high, value))
    32
    33
    34  class ReactiveDataBlock(ModbusSequentialDataBlock):
    35      """A data block that tracks when values are externally written,
    36      and uses those as the new base for drift — instead of silently
    37      overwriting them with the simulator's internal state."""
    38
    39      def __init__(self, address, values):
```

```
33
34  class ReactiveDataBlock(ModbusSequentialDataBlock):
35      """A data block that tracks when values are externally written,
36      and uses those as the new base for drift — instead of silently
37      overwriting them with the simulator's internal state."""
38
39      def __init__(self, address, values):
40          super().__init__(address, values)
41          self._internal = {
42              REG_TEMP: 670,
43              REG_PRESSURE: 340,
44              REG_FLOW: 900,
45          }
46          self._lock = threading.Lock()
47
48      def setValues(self, address, values):
49          with self._lock:
50              for i, val in enumerate(values):
51                  reg = address + i
52                  if reg in self._internal:
53                      self._internal[reg] = val
54          super().setValues(address, values)
55
56      def get_internal(self, reg):
57          with self._lock:
58              return self._internal.get(reg, 0)
59
60      def set_internal(self, reg, val):
61          with self._lock:
62              self._internal[reg] = val
63
64
65  def simulate_normal_operation(store, block, stop_event):
66      while not stop_event.is_set():
67          temp = clamp(block.get_internal(REG_TEMP) + random.randint(-2, 2), *TEMP_RANGE)
68          pressure = clamp(block.get_internal(REG_PRESSURE) + random.randint(-3, 3), *PRESSURE_RANGE)
69          flow = clamp(block.get_internal(REG_FLOW) + random.randint(-10, 10), *FLOW_RANGE)
70
71          block.set_internal(REG_TEMP, temp)
72          block.set_internal(REG_PRESSURE, pressure)
73          block.set_internal(REG_FLOW, flow)
74
```

Guard and alarm logic

```
icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba guard.py | sed -n '1,240p'
     1  import logging
     2  import threading
     3  import time
     4  import json
     5  import os
     6  from collections import deque
     7
     8  from pymodbus.client import ModbusTcpClient
     9  from pymodbus.datastore import (
    10      ModbusSequentialDataBlock,
    11      ModbusServerContext,
    12      ModbusSlaveContext,
    13  )
    14  from pymodbus.server import StartTcpServer
    15
    16  logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
    17  log = logging.getLogger("guard")
    18
    19  PLANT_HOST = "127.0.0.1"
    20  PLANT_PORT = 5020
    21  HOST = "127.0.0.1"
    22  GUARD_PORT = 5021
    23
    24  REG_TEMP = 0
    25  REG_PRESSURE = 1
    26  REG_FLOW = 2
    27  REG_ALARM = 3
    28
    29  TEMP_RANGE     = (600, 750)   # tenths of a degree C
    30  PRESSURE_RANGE = (300, 400)   # tenths of a bar
    31  FLOW_RANGE     = (500, 1500)  # tenths of Nm3/h
    32
    33  MAX_TEMP_STEP      = 30
    34  MAX_PRESSURE_STEP  = 50
    35  MAX_FLOW_STEP      = 100
    36  DRIFT_THRESHOLD_FLOW      = 40  # 4.0 Nm3/h cumulative over the window
    37
    38  EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
    39  event_log_lock = threading.Lock()
    40
    41
    42  def log_event(reason):
    43      event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
    44      with event_log_lock:
    45          try:
    46              with open(EVENT_LOG_FILE, "a") as f:
    47                  f.write(json.dumps(event) + "\n")
    48          except Exception as e:
    49              log.error("Failed to write event log: %s", e)
    50
    51
    52  class GuardedContext(ModbusSlaveContext):
    53      """
    54      A Modbus write-through firewall.
    55
    56      Design rule: this class NEVER accepts a value it hasn't itself validated.
    57      - Absolute range check: hard reject outside physical safe limits.
    58      - Step check: hard reject any single write that moves too far too fast.
    59      - Rolling-baseline drift check: compares the CURRENT accepted value to
    60        the accepted value from N writes ago, so an attacker cannot bypass
    61        detection by taking many small steps.
    62
    63      On any rejection: the write is dropped entirely. Nothing is forwarded
    64      to the plant, nothing is written to the local mirror. The guard's
    65      exposed state (what the guarded dashboard sees) simply does not move.
    66      """
    67
    68      def __init__(self, plant_client, *args, **kwargs):
    69          super().__init__(*args, **kwargs)
    70          self.plant_client = plant_client
    71          self.lock = threading.Lock()
    72          # Rolling baseline histories store ACCEPTED values only, not
    73          # attempted writes. That's what makes this a true baseline of
    74          # "what the plant has actually been allowed to do."
    75          self.temp_accepted_history     = deque(maxlen=DRIFT_WINDOW)
```

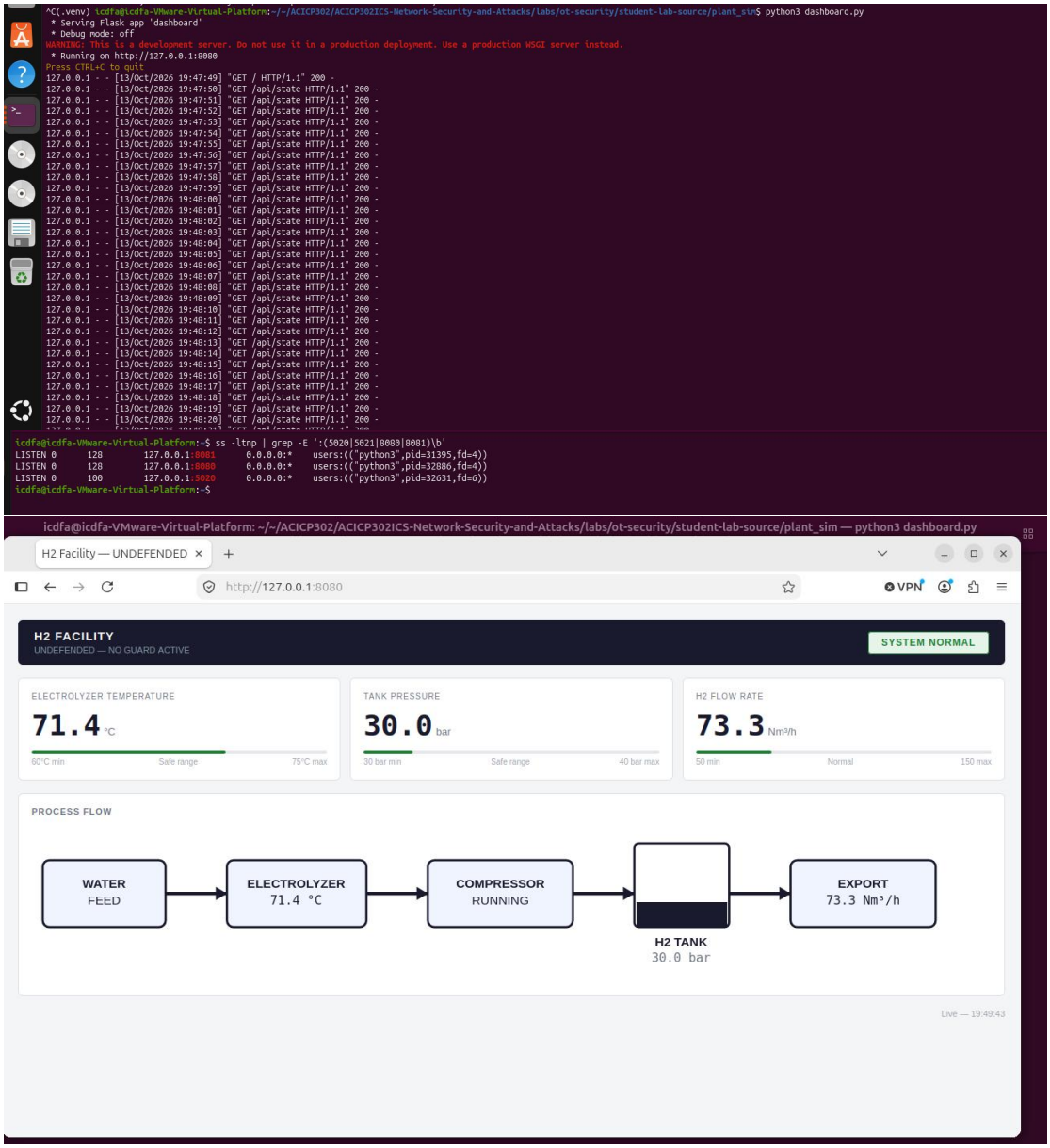
```
164
165         if len(drift_history) == DRIFT_WINDOW:
166             drift = requested - drift_history[0]
167             if abs(drift) > drift_threshold:
168                 reasons.append(
169                     f"cumulative drift of {drift/unit_divisor:.1f}{unit_label} "
170                     f"over last {DRIFT_WINDOW} accepted writes exceeds "
171                     f"{drift_threshold/unit_divisor:.1f}{unit_label} limit"
172                 )
173
174         return reasons
175
176     def setValues(self, fc_as_hex, address, values):
177         requested = values[0]
178         reasons = []
179
180         if address == REG_TEMP:
181             current = self.getValues(3, REG_TEMP, count=1)[0]
182             reasons = self._check(address, requested, current, TEMP_RANGE,
183                                   MAX_TEMP_STEP, self.temp_accepted_history,
184                                   DRIFT_THRESHOLD_TEMP, "C", 10)
185         elif address == REG_PRESSURE:
186             current = self.getValues(3, REG_PRESSURE, count=1)[0]
187             reasons = self._check(address, requested, current, PRESSURE_RANGE,
188                                   MAX_PRESSURE_STEP, self.pressure_accepted_history,
189                                   DRIFT_THRESHOLD_PRESSURE, " bar", 10)
190         elif address == REG_FLOW:
191             current = self.getValues(3, REG_FLOW, count=1)[0]
192             reasons = self._check(address, requested, current, FLOW_RANGE,
193                                   MAX_FLOW_STEP, self.flow_accepted_history,
194                                   DRIFT_THRESHOLD_FLOW, " Nm3/h", 10)
195         else:
196             # Unknown/alarm register — allow passthrough of writes (e.g. resets)
197             with self.lock:
198                 try:
199                     self.plant_client.write_register(address, requested, slave=1)
200                 except Exception as e:
201                     log.error("Forward failed: %s", e)
202             super().setValues(fc_as_hex, address, values)
203             return
```

Dashboard mapping

```
icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba dashboard.py | sed -n '190,230p'
   190  setInterval(poll,1000);poll();
   191  </script>
   192  </body>
   193  </html>
   194  """
   195
   196  app = Flask(__name__)
   197  state = {"temp":0,"pressure":0,"flow":0,"alarm":0,"connected":False}
   198  state_lock = threading.Lock()
   199
   200  def poll_modbus():
   201      client = ModbusTcpClient(TARGET_HOST, port=TARGET_PORT)
   202      while True:
   203          try:
   204              if not client.connect():
   205                  with state_lock: state["connected"] = False
   206                  time.sleep(1); continue
   207              r = client.read_holding_registers(0,4,slave=1)
   208              if not r.isError():
   209                  with state_lock:
   210                      state.update({"temp":r.registers[0],"pressure":r.registers[1],"flow":r.registers[2],"alarm":r.registers[3],"connected":True})
   211          except:
   212              with state_lock: state["connected"] = False
   213          time.sleep(1)
   214
   215  @app.route("/api/state")
   216  def api_state():
   217      with state_lock: return jsonify(dict(state))
   218
   219  @app.route("/")
   220  def index(): return render_template_string(PAGE)
   221
   222  if __name__ == "__main__":
   223      threading.Thread(target=poll_modbus,daemon=True).start()
   224      app.run(host="127.0.0.1",port=8080)
```

Start the normal simulation and dashboard

```
^C(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ python3 plant.py
2026-10-13 19:24:14,801 INFO telemetry temp=67.2C pressure=33.7bar flow=89.2Nm3/h alarm=0
2026-10-13 19:24:14,801 INFO Plant simulator listening on 127.0.0.1:5020 (no protection layer active)
2026-10-13 19:24:14,803 INFO Server listening.
2026-10-13 19:24:15,813 INFO telemetry temp=67.0C pressure=33.5bar flow=89.5Nm3/h alarm=0
2026-10-13 19:24:16,827 INFO telemetry temp=67.2C pressure=33.2bar flow=90.5Nm3/h alarm=0
2026-10-13 19:24:17,835 INFO telemetry temp=67.0C pressure=32.9bar flow=90.1Nm3/h alarm=0
2026-10-13 19:24:18,846 INFO telemetry temp=66.8C pressure=33.2bar flow=90.3Nm3/h alarm=0
2026-10-13 19:24:19,847 INFO telemetry temp=66.7C pressure=32.9bar flow=90.2Nm3/h alarm=0
2026-10-13 19:24:20,856 INFO telemetry temp=66.7C pressure=33.0bar flow=89.3Nm3/h alarm=0
2026-10-13 19:24:21,861 INFO telemetry temp=66.9C pressure=33.1bar flow=88.8Nm3/h alarm=0
2026-10-13 19:24:22,872 INFO telemetry temp=66.7C pressure=32.8bar flow=88.4Nm3/h alarm=0
2026-10-13 19:24:23,873 INFO telemetry temp=66.7C pressure=33.1bar flow=89.3Nm3/h alarm=0
2026-10-13 19:24:24,877 INFO telemetry temp=66.7C pressure=33.2bar flow=88.4Nm3/h alarm=0
2026-10-13 19:24:25,887 INFO telemetry temp=66.9C pressure=33.5bar flow=89.2Nm3/h alarm=0
2026-10-13 19:24:26,888 INFO telemetry temp=67.1C pressure=33.2bar flow=89.9Nm3/h alarm=0
2026-10-13 19:24:27,889 INFO telemetry temp=67.2C pressure=33.3bar flow=89.3Nm3/h alarm=0
2026-10-13 19:24:28,894 INFO telemetry temp=67.3C pressure=33.5bar flow=90.0Nm3/h alarm=0
2026-10-13 19:24:29,901 INFO telemetry temp=67.4C pressure=33.2bar flow=90.0Nm3/h alarm=0
2026-10-13 19:24:30,916 INFO telemetry temp=67.3C pressure=33.3bar flow=90.2Nm3/h alarm=0
2026-10-13 19:24:31,930 INFO telemetry temp=67.2C pressure=33.2bar flow=90.3Nm3/h alarm=0
2026-10-13 19:24:32,934 INFO telemetry temp=67.1C pressure=33.4bar flow=89.4Nm3/h alarm=0
2026-10-13 19:24:33,937 INFO telemetry temp=66.9C pressure=33.7bar flow=90.1Nm3/h alarm=0
2026-10-13 19:24:34,951 INFO telemetry temp=66.8C pressure=33.6bar flow=89.8Nm3/h alarm=0
2026-10-13 19:24:35,963 INFO telemetry temp=66.7C pressure=33.4bar flow=89.5Nm3/h alarm=0
2026-10-13 19:24:36,977 INFO telemetry temp=66.5C pressure=33.1bar flow=90.2Nm3/h alarm=0
2026-10-13 19:24:37,978 INFO telemetry temp=66.7C pressure=33.3bar flow=91.0Nm3/h alarm=0
2026-10-13 19:24:38,981 INFO telemetry temp=66.7C pressure=33.1bar flow=91.8Nm3/h alarm=0
2026-10-13 19:24:39,989 INFO telemetry temp=66.7C pressure=32.8bar flow=91.1Nm3/h alarm=0
2026-10-13 19:24:40,992 INFO telemetry temp=66.5C pressure=32.6bar flow=90.7Nm3/h alarm=0
2026-10-13 19:24:42,003 INFO telemetry temp=66.4C pressure=32.9bar flow=90.2Nm3/h alarm=0
2026-10-13 19:24:43,004 INFO telemetry temp=66.3C pressure=33.1bar flow=90.4Nm3/h alarm=0
2026-10-13 19:24:44,013 INFO telemetry temp=66.5C pressure=33.2bar flow=90.8Nm3/h alarm=0
2026-10-13 19:24:45,028 INFO telemetry temp=66.6C pressure=33.5bar flow=91.3Nm3/h alarm=0
2026-10-13 19:24:46,031 INFO telemetry temp=66.4C pressure=33.8bar flow=91.6Nm3/h alarm=0
2026-10-13 19:24:47,038 INFO telemetry temp=66.2C pressure=34.0bar flow=91.8Nm3/h alarm=0
```



The direct H2 Facility dashboard loaded successfully at http://127.0.0.1:8080. It displayed a system-normal status, an electrolyzer temperature of 72.7 °C, tank pressure of 31.3 bar, and hydrogen flow of 80.9 Nm³/h. The interface identified the direct path as “UNDEFENDED -NO GUARD ACTIVE.” These readings represent one observation of the simulated process; continued monitoring is needed to establish normal operating behaviour over time.

Record your register map

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source$ python3 - <<'PY'
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient("127.0.0.1", port=5020, timeout=3)

try:
    if not client.connect():
        print("Connection failed: plant server is not reachable.")
    else:
        result = client.read_holding_registers(0, count=4, slave=1)
        if result.isError():
            print("Modbus read failed:", result)
        else:
            names = ["Temperature", "Pressure", "Flow", "Alarm"]
            for address, (name, raw) in enumerate(zip(names, result.registers)):
                if address < 3:
                    print(f"Address {address}: {name} = {raw / 10:.1f} (raw={raw})")
                else:
                    print(f"Address {address}: {name} = {raw}")
finally:
    client.close()
PY
Address 0: Temperature = 69.9 (raw=699)
Address 1: Pressure = 30.4 (raw=304)
Address 2: Flow = 70.1 (raw=701)
Address 3: Alarm = 0
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source$
```

A02-03-modbus-register-read

Holding register address	Data point	Raw value observed	Converted value
0	Temperature	699	69.9 °C
1	Pressure	304	30.4 bar
2	Flow rate	701	70.1 Nm³/h
3	Alarm status	0	Inactive

These are the values from the verification at that moment. Because the simulator updates continuously, later readings may differ.

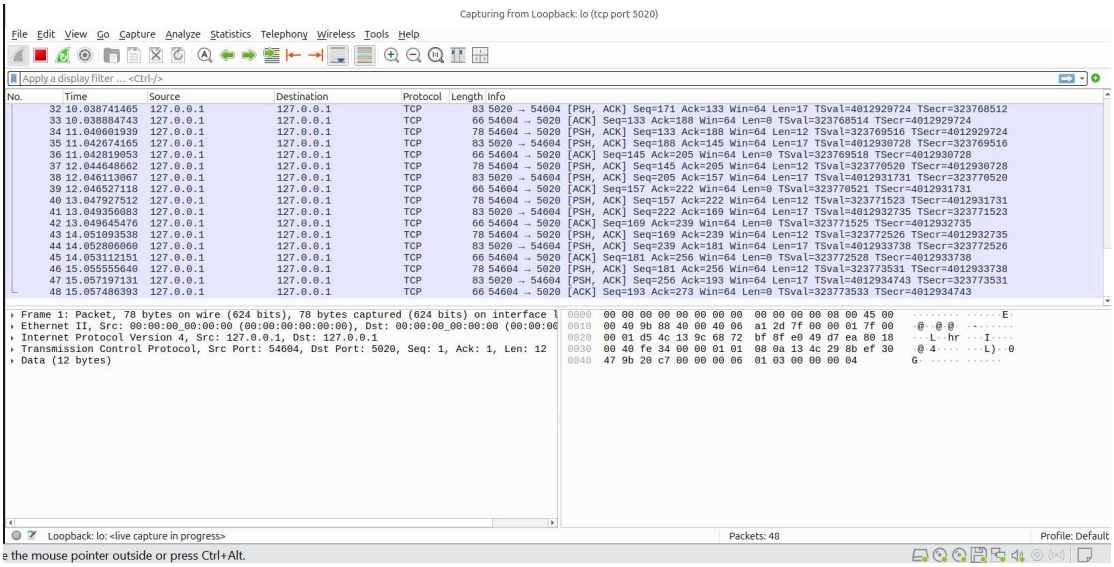
A read-only Modbus/TCP client connected to the plant simulator at 127.0.0.1:5020 and successfully read four holding registers, beginning at zero-based address 0. The observed raw values were 699, 304, 701 and 0. Applying the simulator's scale factor of 10 to the first three registers gave temperature of 69.9 °C, pressure of 30.4 bar and flow rate of 70.1 Nm³/h. Register 3 reported an inactive alarm status (0). The results confirmed successful communication and retrieval of the expected register count during normal simulated operation.

Alarm-rule source/display difference

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba dashboard.py | sed -n '165,190p'
   165      const d=await(await fetch('/api/state')).json();
   166      const temp=d.temp/10, pres=d.pressure/10, flow=d.flow/10;
   167      document.getElementById('v-temp').textContent=temp.toFixed(1);
   168      document.getElementById('v-pres').textContent=pres.toFixed(1);
   169      document.getElementById('v-flow').textContent=flow.toFixed(1);
   170      document.getElementById('diag-temp').textContent=temp.toFixed(1)+' °C';
   171      document.getElementById('diag-pres').textContent=pres.toFixed(1)+' bar';
   172      document.getElementById('diag-flow').textContent=flow.toFixed(1)+' Nm³/h';
   173      document.getElementById('bar-temp').style.width=clamp((temp-55)/25,0,1)*100+'%';
   174      document.getElementById('bar-pres').style.width=clamp((pres-25)/30,0,1)*100+'%';
   175      document.getElementById('bar-flow').style.width=clamp((flow-40)/130,0,1)*100+'%';
   176
   177      const fillFrac = clamp(pres/100, 0, 1);
   178      const maxH = 75, bottomY = 90;
   179      const h = maxH * fillFrac;
   180      document.getElementById('tank-inner').setAttribute('y', bottomY - h);
   181      document.getElementById('tank-inner').setAttribute('height', h);
   182
   183      const alarm = pres > 40 || pres < 28 || temp > 76 || temp < 57 || flow > 156 || flow < 44;
   184      document.getElementById('bar-pres').className = 'card-bar ' + (alarm ? 'bar-alarm' : 'bar-ok');
   185      document.getElementById('bar-temp').className = 'card-bar ' + (alarm ? 'bar-alarm' : 'bar-ok');
   186      setAlarm(alarm);
   187      document.getElementById('footer').textContent=(d.connected?'Live':'Disconnected')+' — '+new Date().toTimeString().slice(0,8);
   188    }catch(e){document.getElementById('footer').textContent='Error: '+e.message;}
   189  }
   190  setInterval(poll,1000);poll();
```

The dashboard acts as the Modbus/TCP client and the plant simulator acts as the server. The client requests four holding registers starting at address 0 using function code 03 (Read Holding Registers). The server responds with the current values of addresses 0-3. The dashboard scales the first three raw values by dividing them by 10 for display and uses the fourth register as the plant alarm status data. The dashboard polls approximately once per second. The observed read-only verification returned four registers successfully without writing to the process.

Activity 03 - Packet Visibility and Protocol Analysis



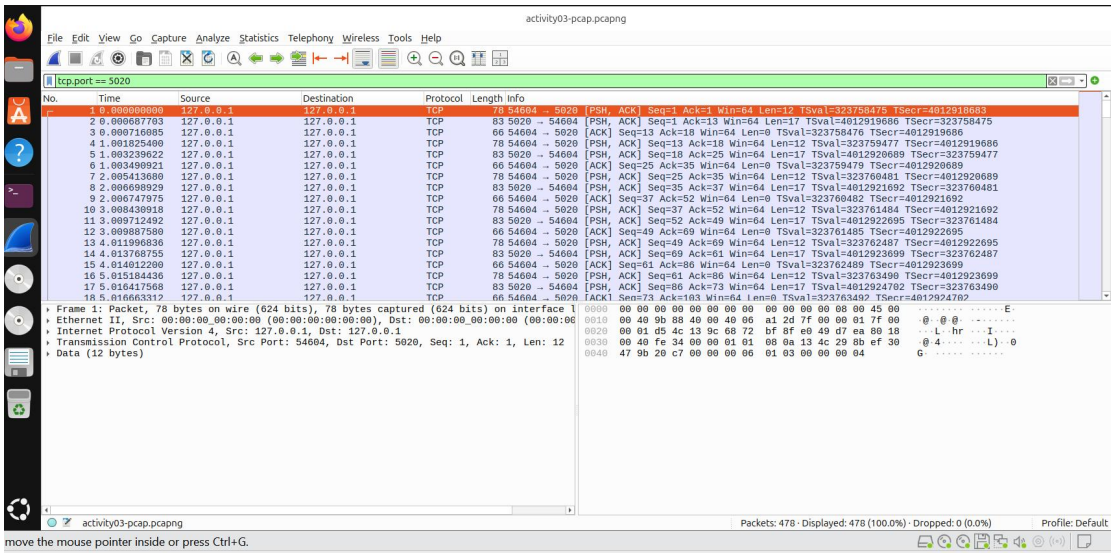
1. Interpret the register values

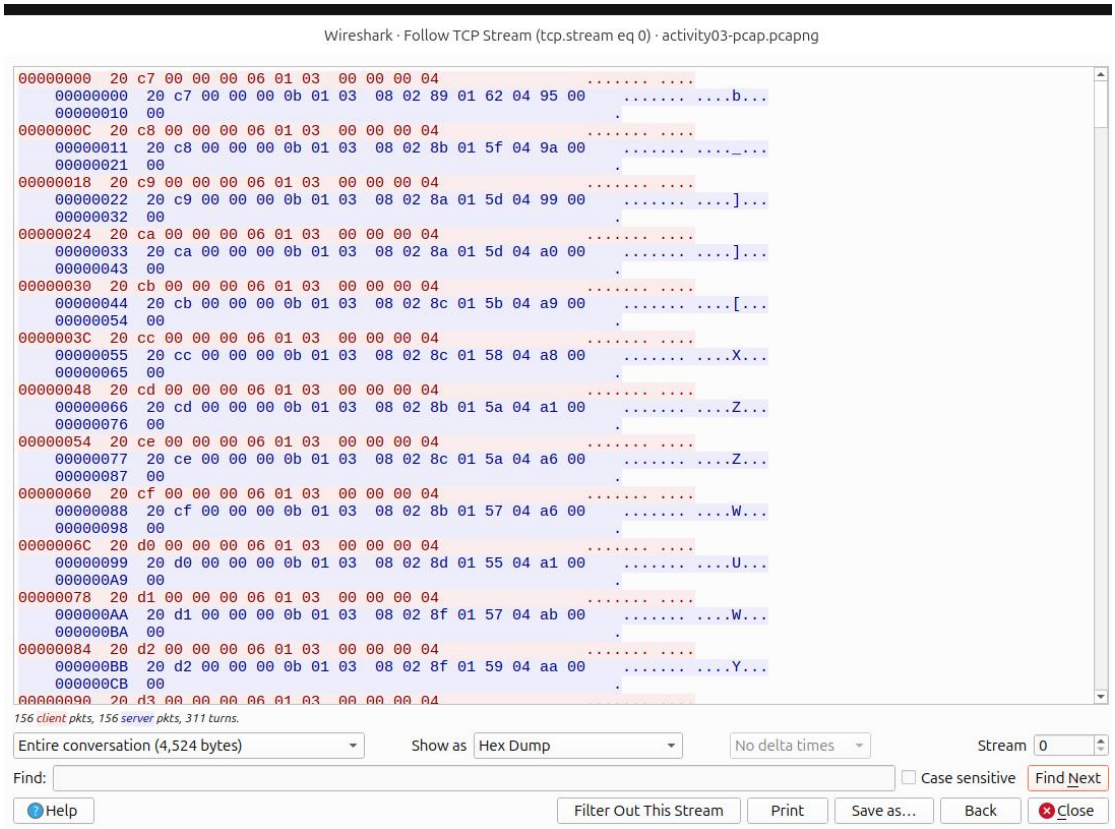
```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ python3 - <<'PY'
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient("127.0.0.1", port=5020, timeout=3)
try:
    if client.connect():
        result = client.read_holding_registers(0, count=4, slave=1)
        if result.isError():
            print("Modbus read error:", result)
        else:
            print("Registers:", result.registers)
    else:
        print("Connection failed")
finally:
    client.close()
PY
Registers: [642, 346, 1161, 0]
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$
```

Based on the lab's register mapping, the first three registers are scaled by 10.

Register address	Raw value	Interpreted value
0 — Temperature	642	64.2 °C
1 — Pressure	346	34.6 bar
2 — Flow	1161	116.1 Nm³/h
3 — Alarm	0	Alarm inactive





Decode



Decoded response:

Register 0: 649 = 64.9 °C

Register 1: 354 = 35.4 bar

Register 2: 1173 = 117.3 Nm³/h

Register 3: 0 = alarm inactive

Validation: Request and response transaction IDs matched (0x20C7). The function codes matched, and the response contained four registers.

In conclusion, the decoded bytes are consistent with a successful Modbus/TCP read transaction. This offline analysis does not independently establish the packet endpoints, timing, external network reachability, or the state of a physical industrial process.

Activity 04 - Safe discovery and exposure mapping

Objective

This activity is to identify and document the local services configured in the supplied ICS simulation by inspecting source-code binding and target settings and performing authorized, read-only checks within the loopback interface. The activity aims to verify documented service endpoints, establish an approved scope of work, and assess each component's role, communication protocol, process significance, and potential consequences of unauthorized access or misuse. It also seeks to develop an evidence-based risk register identifying potential security risks and appropriate preventative and detective controls, while preserving the integrity of the simulation, avoiding external systems, and documenting findings through a scope card, service inventory, command outputs, screenshots, and a concise conclusion.

source configuration

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ grep -n -i -E '127\.0\.0\.1|0\.0\.0\.0|host=|host:|port=|port:|5020|5021|8080|8081' \
  "$SRC/plant_sim/plant.py" \
  "$SRC/plant_sim/dashboard.py" \
  "$SRC/plant_sim/guard.py" \
  "$SRC/plant_sim/dashboard_guarded.py"
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/plant.py:26:HOST = "127.0.0.1"
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/plant.py:27:PORT = 5020
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard.py:6:TARGET_HOST = "127.0.0.1"
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard.py:7:TARGET_PORT = 5020
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard.py:201:    client = ModbusTcpClient(TARGET_HOST, port=TARGET_PORT)
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard.py:224:    app.run(host="127.0.0.1",port=8080)
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/guard.py:19:PLANT_HOST = "127.0.0.1"
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/guard.py:20:PLANT_PORT = 5020
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/guard.py:21:HOST = "127.0.0.1"
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/guard.py:22:GUARD_PORT = 5021
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/guard.py:201:    plant_client = ModbusTcpClient(PLANT_HOST, port=PLANT_PORT)
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard_guarded.py:8:TARGET_HOST = "127.0.0.1"
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard_guarded.py:9:TARGET_PORT = 5021
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard_guarded.py:220:    client = ModbusTcpClient(TARGET_HOST, port=TARGET_PORT)
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard_guarded.py:265:    app.run(host="127.0.0.1",port=8081)
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ {
  nl -ba "$SRC/plant_sim/plant.py" | sed -n '20,35p'
  nl -ba "$SRC/plant_sim/dashboard.py" | sed -n '1,12p;218,228p'
  nl -ba "$SRC/plant_sim/guard.py" | sed -n '15,28p'
  nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '1,12p;258,268p'
} | tee "$EV/04-source-detail.txt"
    20
    21  TEMP_RANGE = (600, 750)
    22  PRESSURE_RANGE = (300, 400)
    23  FLOW_RANGE = (500, 1500)
    24
    25  UPDATE_INTERVAL_SEC = 1.0
    26  HOST = "127.0.0.1"
    27  PORT = 5020
    28
    29
    30  def clamp(value, low, high):
    31      return max(low, min(high, value))
    32
    33
    34  class ReactiveDataBlock(ModbusSequentialDataBlock):
    35      """A data block that tracks when values are externally written,
     1  from flask import Flask, jsonify, render_template_string
     2  from pymodbus.client import ModbusTcpClient
     3  import threading
     4  import time
     5
     6  TARGET_HOST = "127.0.0.1"
     7  TARGET_PORT = 5020
     8
     9  PAGE = """
    10  <!doctype html>
    11  <html>
    12  <head>
   218
   219  @app.route("/")
   220  def index(): return render_template_string(PAGE)
   221
   222  if __name__ == "__main__":
   223      threading.Thread(target=poll_modbus,daemon=True).start()
   224      app.run(host="127.0.0.1",port=8080)
```

Local service inventory

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ {
  echo "ACICP302 Activity 04 — Local Service Inventory"
  echo "VM: $(hostname)"
  echo "Local time: $(date)"
  echo
  echo "Loopback interface:"
  ip -brief addr show lo
  echo
  echo "Listening TCP services:"
  ss -ltnp
} 2>&1 | tee "$EV/04-local-services.txt"
ACICP302 Activity 04 — Local Service Inventory
VM: icdfa-VMware-Virtual-Platform
Local time: Wed Oct 14 11:49:20 AM WAT 2026

Loopback interface:
lo               UNKNOWN        127.0.0.1/8 ::1/128

Listening TCP services:
State  Recv-Q Send-Q Local Address:Port Peer Address:PortProcess
LISTEN 0      128        127.0.0.1:8081      0.0.0.0:*    users:(("python3",pid=31395,fd=4))
LISTEN 0      4096       127.0.0.54:53       0.0.0.0:*
LISTEN 0      4096   127.0.0.53%lo:53        0.0.0.0:*
LISTEN 0      100        127.0.0.1:5020      0.0.0.0:*    users:(("python3",pid=32631,fd=6))
LISTEN 0      4096       127.0.0.1:631       0.0.0.0:*
LISTEN 0      4096           [::1]:631          [::]:*
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$
```

Approved scope card]

```
ACICP302 — Activity 04: Safe Discovery and Exposure Mapping

VM: icdfa-VMware-Virtual-Platform
Activity date: 14 October 2026
Target: Local loopback only (127.0.0.1)
Authorized source: labs/ot-security/student-lab-source/
Permitted methods: Source inspection and read-only local service inventory
Time window: 14 October 2026, approximately 11:41–11:49 WAT
Expected state: Existing lab services remain unchanged
Prohibited: External targets, subnet scans, attack scripts, process-value
changes, and unauthorized service changes
Stop condition: Unexpected listener, non-loopback application binding,
unexpected process behaviour, or discrepancy requiring instructor review
Recovery action: Stop further activity, preserve evidence, and notify the
instructor. Do not restart or alter services without authorization.
```

Risk-register

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ sed -n '1,25p' "$EV/04-risk-register.txt"
ACICP302 — Activity 04 Risk Register
Date: 14 October 2026
Scope: Approved student lab source and local loopback only

R1 — Modbus/TCP process-data exposure
Evidence: plant.py lines 26-27 configure 127.0.0.1:5020.
Inventory: 127.0.0.1:5020 was listening.
Potential consequence: Unauthorized access to process telemetry;
write capability could affect simulated process values if permitted.
Status: Exposure beyond loopback was not observed or tested.
Preventative controls: Keep the service loopback-bound; restrict clients
and write operations according to the approved lab design.
Detective controls: Review application logs and monitor unexpected
Modbus write requests.

R2 — Direct dashboard availability
Evidence: dashboard.py line 224 binds the dashboard to 127.0.0.1:8080.
Inventory: Port 8080 was not listening during the recorded check.
Potential consequence: If enabled without adequate access controls,
an exposed dashboard could disclose process information or permit
unauthorized interaction through available routes.
Status: Not listening at inventory time; route permissions not assessed.
Preventative controls: Loopback binding, authentication where appropriate,
and least privilege.
Detective controls: Review dashboard access and application logs.
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ sed -n '26,50p' "$EV/04-risk-register.txt"

R3 — Guard service and guarded dashboard
Evidence: guard.py lines 19-22 configure the plant target at
127.0.0.1:5020 and guard endpoint at 127.0.0.1:5021.
dashboard_guarded.py lines 8-9 target 127.0.0.1:5021;
line 265 binds the dashboard to 127.0.0.1:8081.
Inventory: Port 8081 was listening; port 5021 was not listening.
Potential consequence: If guard rules are incomplete or bypassable,
unauthorized requests might not be adequately controlled.
Status: Guard-rule completeness and enforcement were not tested.
Preventative controls: Validate authorization and safety checks for
all relevant operations; retain loopback binding.
Detective controls: Review event logs and record rejected requests.

R4 — Service-binding drift
Evidence: The four application configurations specify loopback addresses.
Inventory: Observed application listeners on 5020 and 8081 were loopback-
bound. Ports 8080 and 5021 were not listening at the recorded time.
Potential consequence: A future broad bind could expose lab services
outside the intended boundary.
Status: No non-loopback application listener among the observed services.
Preventative controls: Keep explicit loopback bindings and isolate the VM.
Detective controls: Recheck listener addresses and VM network settings.

Conclusion:
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ tail -n 12 "$EV/04-risk-register.txt"
Potential consequence: A future broad bind could expose lab services
outside the intended boundary.
Status: No non-loopback application listener among the observed services.
Preventative controls: Keep explicit loopback bindings and isolate the VM.
Detective controls: Recheck listener addresses and VM network settings.

Conclusion:
The source configuration reviewed is loopback-bound. At the recorded
inventory time, the plant service on 5020 and guarded dashboard on 8081
were listening; 8080 and 5021 were not. This is a time-specific snapshot.
No external scanning, attack execution, process-value changes, or
unauthorized service modifications were performed for this activity.
```

Observation:

Activity 04 was completed using the approved student lab source within the Ubuntu virtual machine. The activity focused on source-code inspection, verification of local network interfaces, read-only TCP service inventory, and assessment of potential security risks within the authorized loopback environment.

Source inspection confirmed that the plant Modbus/TCP service was configured on 127.0.0.1:5020, the direct dashboard on 127.0.0.1:8080, the guard endpoint on 127.0.0.1:5021, and the guarded dashboard on 127.0.0.1:8081.

The recorded service inventory showed listeners on 127.0.0.1:5020 and 127.0.0.1:8081. Ports 8080 and 5021 were not listening at the time of observation. The findings were documented in the local service inventory, source configuration evidence, approved scope card, and risk register.

The risk assessment identified potential concerns involving unauthorized Modbus access, dashboard access controls, guard-rule completeness, and unintended service exposure. Preventative and detective controls were recommended based on the available evidence. All required screenshots and supporting text files were saved in the Activity 04 evidence folder.

The activity was conducted using source inspection and read-only local checks. No external scanning, attack scripts, process-value modifications, or unauthorized service changes were performed.

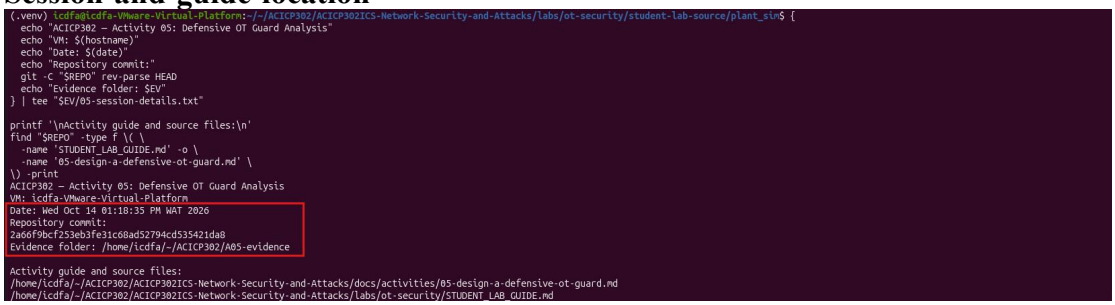
Conclusion: Activity 04 established a documented baseline of the lab's configured endpoints and observed local services. The evidence and risk register provide a foundation for the next practical activity.

Activity 05 - Analyze the supplied defensive OT guard

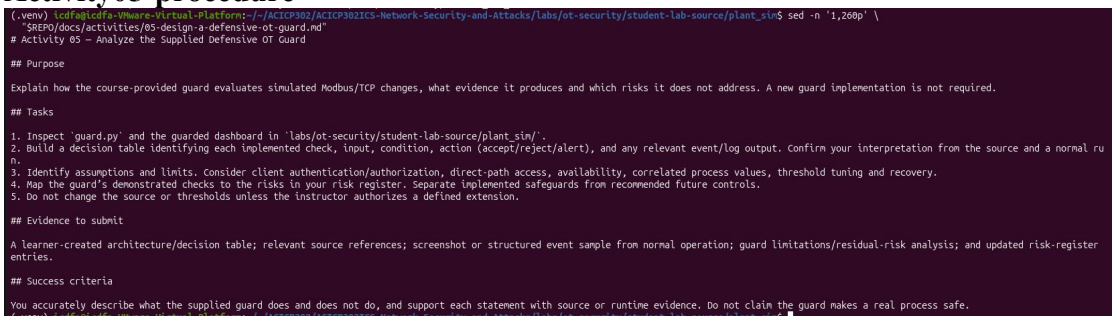
Objective

This activity is to examine and understand the existing defensive Operational Technology (OT) guard provided in the canonical student source by inspecting guard.py and the guarded dashboard. The activity aims to document the guard's architecture, validation checks, decision conditions, actions, and associated audit logs or event records using a decision table, and to verify its behavior through a normal simulation run and instructor-approved scenarios. It also seeks to relate the demonstrated defensive checks to the risks identified in the risk register and critically assess the guard's limitations, including client authentication and authorization, direct-path access, service availability, correlated process values, threshold tuning, and recovery procedures. All findings will be supported by source references and observed evidence, without modifying the supplied source code or guard thresholds unless explicitly authorized by the instructor.

Session-and-guide-location



Activity05-procedure



student-lab-guide

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ sed -n '1,260p' "$LAB/STUDENT_LAB_GUIDE.md"
# Student Lab Guide — Supplied OT Security Simulation on Ubuntu

**Course:** ACICP302ICS — Network Security & Attacks
**Guide prepared by:** Aminu Idris, AMCPN

This course uses the instructor-provided simulation source. Run only `labs/ot-security/student-lab-source/` inside the assigned Ubuntu guest VM. The `labs/ot-security/archive-original/` directory is an ar
chival copy with unsafe network defaults and **must not be executed**. You are expected to inspect and use the supplied code; building a replacement simulator is not required. Your assessed work is your o
wn analysis, observations, captures, logs, report and evidence.

## 1. Non-negotiable safety rules

- Use only the canonical `student-lab-source/` directory identified in this guide.
- Do not run or copy commands from `archive-original/`.
- Keep all simulation traffic on `127.0.0.1` inside the assigned Ubuntu VM. Do not use bridged networking, port forwarding, real devices, public/campus/home networks or another learner's VM.
- Do not change targets, bind addresses, ports, request counts, durations or thread counts in the supplied scripts.
- If a target/bind address is not `127.0.0.1`, stop and notify the instructor. Do not edit the source to force the check to pass.
- This is an educational simulation, not a production security or process-safety product.

## 2. Prepare the Ubuntu VM

Use the instructor-approved Ubuntu LTS image and VMware or VirtualBox installation. Create a fresh guest VM for this lab and take a clean snapshot. Never install or run this lab on a production OT compute
r.

**VirtualBox:** In VM Settings → Network, do not select Bridged Adapter and do not configure port forwarding. Use NAT temporarily only if the instructor permits it for package/repository downloads. Shut d
own the VM and set the adapter to Not attached before running the lab. See the [VirtualBox networking manual](https://www.virtualbox.org/manual/ch06.html).

**VMware Workstation/Player:** Do not select Bridged and do not configure port forwarding. Use NAT temporarily only for approved downloads. Disconnect the virtual adapter before the lab run. If the instru
ctor requires an attached adapter, use an approved Host-only configuration, never Bridged. See [VMware Host-only networking](https://techdocs.broadcom.com/us/en/vmware-cis/desktop-hypervisors/workstation-
pro/17-0/using-vmware-workstation-pro/configuring-network-connections/configuring-host-only-networking.html) and [networking modes](https://techdocs.broadcom.com/us/en/vmware-cis/desktop-hypervisors/works
tation-pro/17-0/using-vmware-workstation-pro/configuring-network-connections/understanding-common-networking-configurations.html).

The simulation itself uses Linux loopback and does not need internet access. Complete approved downloads first, shut down the guest, disconnect its adapter, then start the lab.

## 3. Obtain the course repository

The GitHub repository is private. Ask the course owner to grant you access, or use the instructor-provided approved archive. Do not share passwords, access tokens or credentials.

If repository access is authorized, use NAT only temporarily for cloning and package setup. In an Ubuntu terminal:

| Script | Demonstration | Supplied command pair |
|---|---|---|
| `attack_hold.py` | Requests a pressure value outside the simulated range | `python3 attack_hold.py 5020` then `python3 attack_hold.py 5021` |
| `false_data_attack.py` | Sends the provided batches of out-of-range process values | `python3 false_data_attack.py 5020` then `python3 false_data_attack.py 5021` |
| `slow_drift_attack.py` | Sends the provided incremental pressure changes | `python3 slow_drift_attack.py 5020` then `python3 slow_drift_attack.py 5021` |
| `replay_attack.py` | Repeats the provided pressure write | `python3 replay_attack.py 5020` then `python3 replay_attack.py 5021` |
| `recon_scan.py` | Reads the limited register range documented by the script | `python3 recon_scan.py 5020` then `python3 recon_scan.py 5021` |
| `dos_attack.py` | Fixed demonstration of 200 read requests using 10 threads | Run only if the instructor approves; then use `python3 dos_attack.py 5020` followed by `python3 dos_attack.py 5021` |

Do not increase request counts, duration, rate or thread count. Run any availability/load demonstration only if it is explicitly approved. After each scenario, record actual script output, dashboard obser
vations and relevant `plant_sim/events.jsonl` entries. Results may vary with timing; report what occurred rather than trying to force a particular result.

## 9. Analysis and evidence

For each scenario, explain which check is relevant (for example, absolute range, step-size or rolling-baseline drift); whether the direct/guarded views and event record agree; and what the result does and
 does not show. Consider limits such as authentication, client authorization, availability, correlated process values, direct-path bypass and process-specific thresholds. This guard is not a real safety s
ystem.

Use the course templates for the evidence log, topology/register map, risk register, test plan and final report. Record the script, local port/path, starting state, expected and observed results, evidence
 reference, conclusion and recovery. Attribute the course source and record its commit; your analysis and evidence must be your own. Do not commit `.venv`, credentials, personal/site data or unapproved lo
gs/captures to the shared course-source repository.

## 10. Stop and clean up

When finished:

1. Stop each service with **Ctrl+C** in its terminal and wait for each demonstration script to finish.
2. Confirm the lab processes have stopped and the local ports are no longer listening.
3. Deactivate the virtual environment with `deactivate`.
4. Handle `plant_sim/events.jsonl` and evidence according to the instructor's policy.
5. Restore the clean VM snapshot if directed and leave the network adapter disconnected.

If a dashboard disconnects, check that the plant was started first and the guard second. If any target/bind check fails, stop; do not edit the source or run the archived scripts.

## Official VM networking references

- [VirtualBox User Manual — Virtual Networking](https://www.virtualbox.org/manual/ch06.html)
- [VMware Workstation — Host-Only Networking](https://techdocs.broadcom.com/us/en/vmware-cis/desktop-hypervisors/workstation-pro/17-0/using-vmware-workstation-pro/configuring-network-connections/configuri
ng-host-only-networking.html)
- [VMware Workstation — Common Networking Configurations](https://techdocs.broadcom.com/us/en/vmware-cis/desktop-hypervisors/workstation-pro/17-0/using-vmware-workstation-pro/configuring-network-connectio
ns/understanding-common-networking-configurations.html)
```

Inspect the guard

Configuration and guard rules

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/guard.py" | sed -n '19,60p'
    19  PLANT_HOST = "127.0.0.1"
    20  PLANT_PORT = 5020
    21  HOST = "127.0.0.1"
    22  GUARD_PORT = 5021
    23
    24  REG_TEMP = 0
    25  REG_PRESSURE = 1
    26  REG_FLOW = 2
    27  REG_ALARM = 3
    28
    29  TEMP_RANGE     = (600, 750)   # tenths of a degree C
    30  PRESSURE_RANGE = (300, 400)   # tenths of a bar
    31  FLOW_RANGE     = (500, 1500)  # tenths of Nm3/h
    32
    33  MAX_TEMP_STEP      = 30
    34  MAX_PRESSURE_STEP  = 50
    35  MAX_FLOW_STEP      = 100
    36
    37  # --- Rolling-baseline drift detection ---
    38  # Instead of comparing write N to write N-1 (which a slow-ramp attack can
    39  # always stay under), we compare the CURRENT accepted value to the accepted
    40  # value from DRIFT_WINDOW writes ago. This catches cumulative creep even
    41  # when every individual step looks small.
    42  DRIFT_WINDOW             = 3   # was 6 — shrinking cuts the attacker's runway in half
    43  DRIFT_THRESHOLD_TEMP     = 20  # 2.0C cumulative over the window
    44  DRIFT_THRESHOLD_PRESSURE = 20  # 2.0 bar cumulative over the window
    45  DRIFT_THRESHOLD_FLOW     = 40  # 4.0 Nm3/h cumulative over the window
    46
    47  EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
    48  event_log_lock = threading.Lock()
    49
    50
    51  def log_event(reason):
    52      event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
    53      with event_log_lock:
    54          try:
    55              with open(EVENT_LOG_FILE, "a") as f:
    56                  f.write(json.dumps(event) + "\n")
    57          except Exception as e:
    58              log.error("Failed to write event log: %s", e)
    59
```

Validation and write decisions

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/guard.py" | sed -n '115,175p'
   115
   116      def setValues(self, fc_as_hex, address, values):
   117          requested = values[0]
   118          reasons = []
   119
   120          if address == REG_TEMP:
   121              current = self.getValues(3, REG_TEMP, count=1)[0]
   122              reasons = self._check(address, requested, current, TEMP_RANGE,
   123                                    MAX_TEMP_STEP, self.temp_accepted_history,
   124                                    DRIFT_THRESHOLD_TEMP, "C", 10)
   125          elif address == REG_PRESSURE:
   126              current = self.getValues(3, REG_PRESSURE, count=1)[0]
   127              reasons = self._check(address, requested, current, PRESSURE_RANGE,
   128                                    MAX_PRESSURE_STEP, self.pressure_accepted_history,
   129                                    DRIFT_THRESHOLD_PRESSURE, " bar", 10)
   130          elif address == REG_FLOW:
   131              current = self.getValues(3, REG_FLOW, count=1)[0]
   132              reasons = self._check(address, requested, current, FLOW_RANGE,
   133                                    MAX_FLOW_STEP, self.flow_accepted_history,
   134                                    DRIFT_THRESHOLD_FLOW, " Nm3/h", 10)
   135          else:
   136              # Unknown/alarm register — allow passthrough of writes (e.g. resets)
   137              with self.lock:
   138                  try:
   139                      self.plant_client.write_register(address, requested, slave=1)
   140                  except Exception as e:
   141                      log.error("Forward failed: %s", e)
   142              super().setValues(fc_as_hex, address, values)
   143              return
   144
   145          if reasons:
   146              reason_text = "; ".join(reasons)
   147              log.warning("BLOCKED write to reg %s: %s", address, reason_text)
   148              log_event(reason_text)
   149              # Option A behavior: reject completely. Do not touch the plant,
   150              # do not touch the local mirror, do not append to drift history.
   151              # The last known-safe accepted value remains authoritative.
   152              return
```

Server and forwarding behaviour

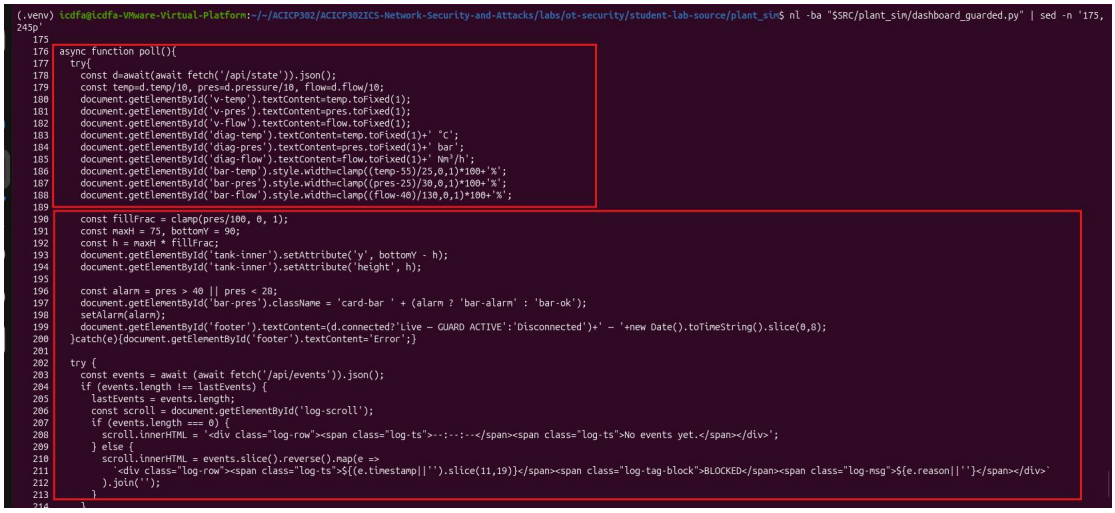
```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/guard.py" | sed -n '175,215p'
   175      absolute safe range. This keeps normal plant drift (temperature
   176      wandering a bit under real physics) visible on the guarded dashboard,
   177      while guaranteeing an attacker who writes directly to the plant's own
   178      port (bypassing the guard entirely) can never have those corrupted
   179      values bleed into the guard's mirror.
   180      """
   181      while not stop_event.is_set():
   182          with lock:
   183              try:
   184                  result = plant_client.read_holding_registers(0, 4, slave=1)
   185                  if not result.isError():
   186                      regs = result.registers
   187                      t, p, f = regs[REG_TEMP], regs[REG_PRESSURE], regs[REG_FLOW]
   188                      if (TEMP_RANGE[0]     <= t <= TEMP_RANGE[1] and
   189                          PRESSURE_RANGE[0] <= p <= PRESSURE_RANGE[1] and
   190                          FLOW_RANGE[0]     <= f <= FLOW_RANGE[1]):
   191                          ModbusSlaveContext.setValues(store, 3, 0, list(regs))
   192                          store.temp_accepted_history.append(t)
   193                          store.pressure_accepted_history.append(p)
   194                          store.flow_accepted_history.append(f)
   195              except Exception as e:
   196                  log.error("Sync error: %s", e)
   197          time.sleep(1.0)
   198
   199
   200  def main():
   201      plant_client = ModbusTcpClient(PLANT_HOST, port=PLANT_PORT)
   202      plant_client.connect()
   203
   204      block = ModbusSequentialDataBlock(0, [670, 340, 900, 0] + [0] * 96)
   205      store = GuardedContext(plant_client, hr=block, zero_mode=True)
   206      context = ModbusServerContext(slaves=store, single=True)
   207
   208      stop_event = threading.Event()
   209      sync_thread = threading.Thread(
   210          target=sync_safe_reads,
   211          args=(store, plant_client, store.lock, stop_event),
   212          daemon=True,
   213      )
   214      sync_thread.start()
```

Guarded dashboard

Guard connection

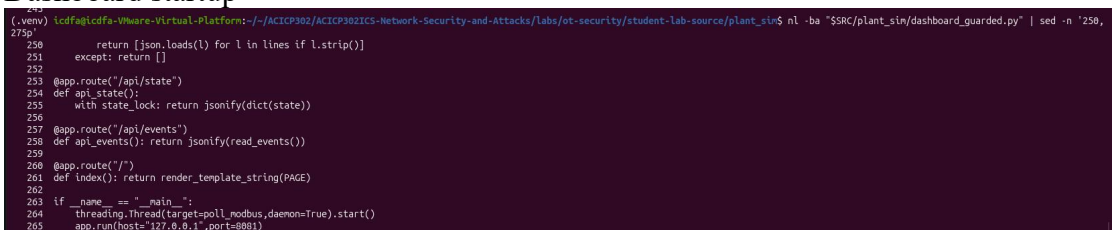
```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '1,45
p'
     1  from flask import Flask, jsonify, render_template_string
     2  from pymodbus.client import ModbusTcpClient
     3  import threading
     4  import time
     5  import json
     6  import os
     7
     8  TARGET_HOST = "127.0.0.1"
     9  TARGET_PORT = 5021
    10  EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
    11
    12  PAGE = """
    13  <!doctype html>
    14  <html>
    15  <head>
    16  <meta charset="utf-8">
    17  <title>H2 Facility — GUARDED</title>
    18  <style>
    19    * { box-sizing: border-box; margin: 0; padding: 0; }
    20    body { font-family: Arial, sans-serif; background: #f4f5f7; color: #1a1a2e; padding: 20px; }
    21    .top-bar { display: flex; justify-content: space-between; align-items: center; background: #1e7e34; color: white; padding: 12px 20px; border-radius: 6px; margin-bottom: 16px; }
    22    .top-bar h1 { font-size: 15px; font-weight: bold; letter-spacing: 1px; }
    23    .top-bar .sub { font-size: 11px; color: #c8e6c9; margin-top: 2px; }
    24    .badge { padding: 6px 16px; border-radius: 4px; font-size: 12px; font-weight: bold; }
    25    .badge-ok { background: white; color: #1e7e34; }
    26    .badge-alarm { background: #c0392b; color: white; }
    27    .grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; margin-bottom: 16px; }
    28    .card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; }
    29    .card-label { font-size: 10px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 8px; }
    30    .card-value { font-size: 36px; font-weight: bold; color: #1a1a2e; font-family: monospace; }
    31    .card-unit { font-size: 13px; color: #6b7280; margin-left: 4px; }
    32    .card-bar-bg { height: 5px; background: #e5e7eb; border-radius: 3px; margin-top: 10px; }
    33    .card-bar { height: 5px; border-radius: 3px; transition: width 0.5s, background 0.5s; }
    34    .bar-ok { background: #1e7e34; }
    35    .bar-alarm { background: #c0392b; }
    36    .card-range { display: flex; justify-content: space-between; font-size: 10px; color: #9ca3af; margin-top: 3px; }
    37    .plant-card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; margin-bottom: 16px; }
    38    .plant-card h3 { font-size: 11px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 12px; }
    39    .log-card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; margin-top: 16px; }
    40    .log-card h3 { font-size: 11px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 10px; }
```

Display and polling logic



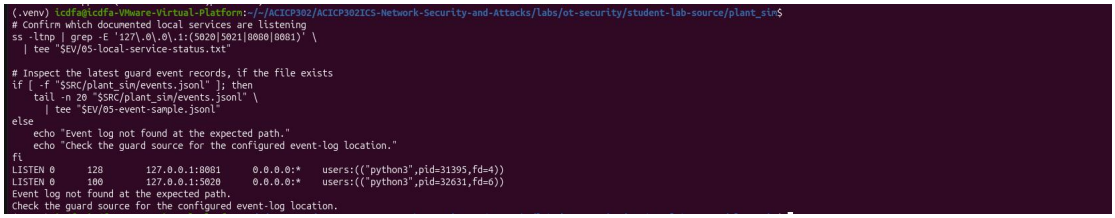
```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '175,245p'
   175
   176  async function poll(){
   177    try{
   178      const d=await(await fetch('/api/state')).json();
   179      const temp=d.temp/10, pres=d.pressure/10, flow=d.flow/10;
   180      document.getElementById('v-temp').textContent=temp.toFixed(1);
   181      document.getElementById('v-pres').textContent=pres.toFixed(1);
   182      document.getElementById('v-flow').textContent=flow.toFixed(1);
   183      document.getElementById('diag-temp').textContent=temp.toFixed(1)+' °C';
   184      document.getElementById('diag-pres').textContent=pres.toFixed(1)+' bar';
   185      document.getElementById('diag-flow').textContent=flow.toFixed(1)+' Nm³/h';
   186      document.getElementById('bar-temp').style.width=clamp((temp-55)/25,0,1)*100+'%';
   187      document.getElementById('bar-pres').style.width=clamp((pres-25)/30,0,1)*100+'%';
   188      document.getElementById('bar-flow').style.width=clamp((flow-40)/130,0,1)*100+'%';
   189
   190      const fillFrac = clamp(pres/100, 0, 1);
   191      const maxH = 75, bottomY = 90;
   192      const h = maxH * fillFrac;
   193      document.getElementById('tank-inner').setAttribute('y', bottomY - h);
   194      document.getElementById('tank-inner').setAttribute('height', h);
   195
   196      const alarm = pres > 40 || pres < 28;
   197      document.getElementById('bar-pres').className = 'card-bar ' + (alarm ? 'bar-alarm' : 'bar-ok');
   198      setAlarm(alarm);
   199      document.getElementById('footer').textContent=(d.connected?'Live — GUARD ACTIVE':'Disconnected')+' — '+new Date().toTimeString().slice(0,8);
   200    }catch(e){document.getElementById('footer').textContent='Error';}
   201
   202    try {
   203      const events = await (await fetch('/api/events')).json();
   204      if (events.length !== lastEvents) {
   205        lastEvents = events.length;
   206        const scroll = document.getElementById('log-scroll');
   207        if (events.length === 0) {
   208          scroll.innerHTML = '<div class="log-row"><span class="log-ts">--:--:--</span><span class="log-ts">No events yet.</span></div>';
   209        } else {
   210          scroll.innerHTML = events.slice().reverse().map(e =>
   211            `<div class="log-row"><span class="log-ts">${(e.timestamp||'').slice(11,19)}</span><span class="log-tag-block">BLOCKED</span><span class="log-msg">${e.reason||''}</span></div>`
   212          ).join('');
   213        }
   214      }
```

Dashboard startup



```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '250,275p'
   250          return [json.loads(l) for l in lines if l.strip()]
   251      except: return []
   252
   253  @app.route("/api/state")
   254  def api_state():
   255      with state_lock: return jsonify(dict(state))
   256
   257  @app.route("/api/events")
   258  def api_events(): return jsonify(read_events())
   259
   260  @app.route("/")
   261  def index(): return render_template_string(PAGE)
   262
   263  if __name__ == "__main__":
   264      threading.Thread(target=poll_modbus,daemon=True).start()
   265      app.run(host="127.0.0.1",port=8081)
```

normal-dashboard



```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$
# Confirm which documented local services are listening
ss -ltnp | grep -E '127\.0\.0\.1:(5020|5021|8080|8081)' \
  | tee "$EV/05-local-service-status.txt"

# Inspect the latest guard event records, if the file exists
if [ -f "$SRC/plant_sim/events.jsonl" ]; then
    tail -n 20 "$SRC/plant_sim/events.jsonl" \
      | tee "$EV/05-event-sample.jsonl"
else
    echo "Event log not found at the expected path."
    echo "Check the guard source for the configured event-log location."
fi
LISTEN 0      128        127.0.0.1:8081      0.0.0.0:*    users:(("python3",pid=31395,fd=4))
LISTEN 0      100        127.0.0.1:5020      0.0.0.0:*    users:(("python3",pid=32631,fd=6))
Event log not found at the expected path.
Check the guard source for the configured event-log location.
```

Service	Observation	Interpretation
Direct Modbus plant — 127.0.0.1:5020	Listening	The simulated plant is running.
Guarded dashboard — 127.0.0.1:8081	Listening	The guarded dashboard process is running.
Guard — 127.0.0.1:5021	Not shown	No listener appeared in this output; we need to investigate.
Direct dashboard — 127.0.0.1:8080	Not shown	No listener appeared in this output.
Event log	Not found at the expected path	We need to locate the configured log path before concluding that event logging is unavailable.

event-log-investigation

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$
# Find the event-log configuration in the supplied source
grep -n -C 3 'EVENT_LOG_FILE' "$SRC/plant_sim/guard.py"

# Find event/logging operations in the guard
grep -nEi 'event|jsonl|logging|write|append|open\(' \
  "$SRC/plant_sim/guard.py"

# Check whether the event log exists anywhere in the canonical source
find "$SRC/plant_sim" -maxdepth 2 -type f \
  \( -name 'events.jsonl' -o -iname '*event*log*' \) -print

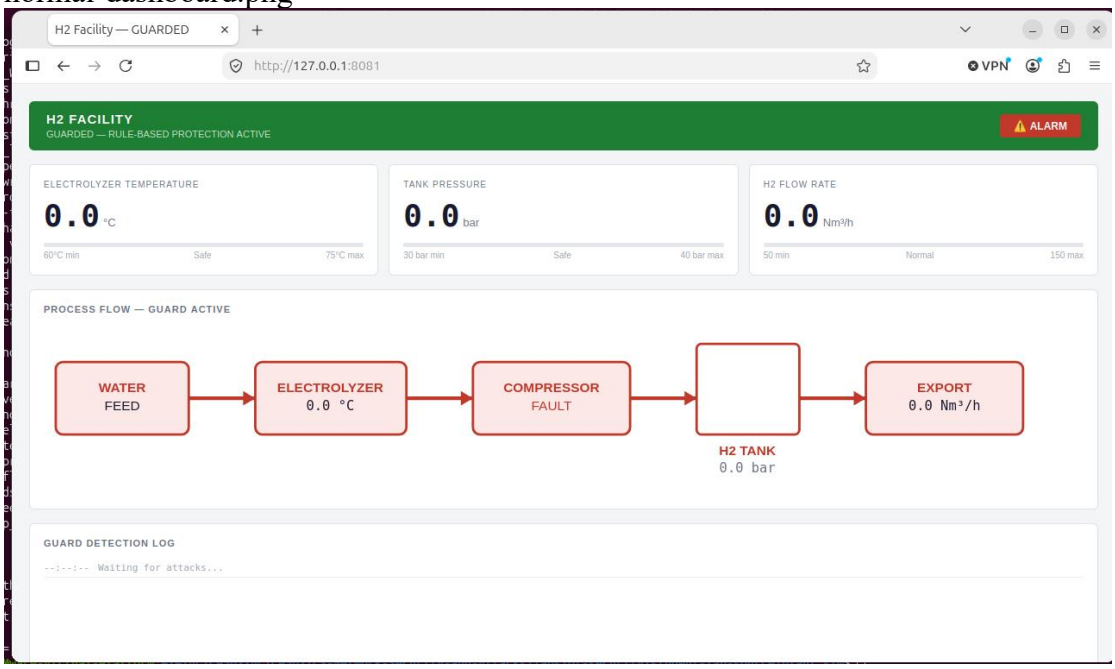
# Save the findings
{
  echo "=== Event-log configuration ==="
  grep -n -C 3 'EVENT_LOG_FILE' "$SRC/plant_sim/guard.py"
  echo
  echo "=== Event/logging operations ==="
  grep -nEi 'event|jsonl|logging|write|append|open\(' \
    "$SRC/plant_sim/guard.py"
  echo
  echo "=== Matching files ==="
  find "$SRC/plant_sim" -maxdepth 2 -type f \
    \( -name 'events.jsonl' -o -iname '*event*log*' \) -print
} | tee "$EV/05-event-log-investigation.txt"
44-DRIFT_THRESHOLD_PRESSURE  = 20  # 2.0 bar cumulative over the window
45-DRIFT_THRESHOLD_FLOW      = 40  # 4.0 Nm3/h cumulative over the window
46-
47:EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
48-event_log_lock = threading.Lock()
49-
50-
--
52-    event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
53-    with event_log_lock:
54-        try:
55:            with open(EVENT_LOG_FILE, "a") as f:
56-                f.write(json.dumps(event) + "\n")
57-        except Exception as e:
58-            log.error("Failed to write event log: %s", e)
1:import logging
16:logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
17:log = logging.getLogger("guard")
38:# Instead of comparing write N to write N-1 (which a slow-ramp attack can
40:# value from DRIFT_WINDOW writes ago. This catches cumulative creep even
47:EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
48:event_log_lock = threading.Lock()
51:def log_event(reason):
52:    event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
53:    with event_log_lock:
55:            with open(EVENT_LOG_FILE, "a") as f:
56:                f.write(json.dumps(event) + "\n")
58:            log.error("Failed to write event log: %s", e)
63:    A Modbus write-through firewall.
67:    - Step check: hard reject any single write that moves too far too fast.
69:      the accepted value from N writes ago, so an attacker cannot bypass
72:    On any rejection: the write is dropped entirely. Nothing is forwarded
82:        # attempted writes. That's what makes this a true baseline of
93:            reasons.append(
100:            reasons.append(
108:                reasons.append(
110:                    f"over last {DRIFT_WINDOW} accepted writes exceeds "
136:            # Unknown/alarm register — allow passthrough of writes (e.g. resets)
139:                self.plant_client.write_register(address, requested, slave=1)
147:            log.warning("BLOCKED write to reg %s: %s", address, reason_text)
148:            log_event(reason_text)
150:            # do not touch the local mirror, do not append to drift history.
158:                self.plant_client.write_register(address, requested, slave=1)
164:            self.temp_accepted_history.append(requested)
166:            self.pressure_accepted_history.append(requested)
168:            self.flow_accepted_history.append(requested)
171:def sync_safe_reads(store, plant_client, lock, stop_event):
177:    while guaranteeing an attacker who writes directly to the plant's own
181:    while not stop_event.is_set():
192:                        store.temp_accepted_history.append(t)
193:                        store.pressure_accepted_history.append(p)
194:                        store.flow_accepted_history.append(f)
208:    stop_event = threading.Event()
211:        args=(store, plant_client, store.lock, stop_event),
223:        stop_event.set()  …      .           .    .…     ..
223:        stop_event.set()
=== Event-log configuration ===
44-DRIFT_THRESHOLD_PRESSURE  = 20  # 2.0 bar cumulative over the window
45-DRIFT_THRESHOLD_FLOW      = 40  # 4.0 Nm3/h cumulative over the window
46-
47:EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
48-event_log_lock = threading.Lock()
49-
50-
--
52-    event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
53-    with event_log_lock:
54-        try:
55:            with open(EVENT_LOG_FILE, "a") as f:
56-                f.write(json.dumps(event) + "\n")
57-        except Exception as e:
58-            log.error("Failed to write event log: %s", e)

=== Event/logging operations ===
1:import logging
16:logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
17:log = logging.getLogger("guard")
38:# Instead of comparing write N to write N-1 (which a slow-ramp attack can
40:# value from DRIFT_WINDOW writes ago. This catches cumulative creep even
47:EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
48:event_log_lock = threading.Lock()
51:def log_event(reason):
52:    event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
53:    with event_log_lock:
55:            with open(EVENT_LOG_FILE, "a") as f:
56:                f.write(json.dumps(event) + "\n")
58:            log.error("Failed to write event log: %s", e)
63:    A Modbus write-through firewall.
67:    - Step check: hard reject any single write that moves too far too fast.
69:      the accepted value from N writes ago, so an attacker cannot bypass
72:    On any rejection: the write is dropped entirely. Nothing is forwarded
82:        # attempted writes. That's what makes this a true baseline of
93:            reasons.append(
```

```
17:log = logging.getLogger("guard")
38:# Instead of comparing write N to write N-1 (which a slow-ramp attack can
40:# value from DRIFT_WINDOW writes ago. This catches cumulative creep even
47:EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
48:event_log_lock = threading.Lock()
51:def log_event(reason):
52:    event = {"timestamp": time.strftime("%Y-%m-%d %H:%M:%S"), "reason": reason}
53:    with event_log_lock:
55:            with open(EVENT_LOG_FILE, "a") as f:
56:                f.write(json.dumps(event) + "\n")
58:            log.error("Failed to write event log: %s", e)
63:    A Modbus write-through firewall.
67:    - Step check: hard reject any single write that moves too far too fast.
69:      the accepted value from N writes ago, so an attacker cannot bypass
72:    On any rejection: the write is dropped entirely. Nothing is forwarded
82:        # attempted writes. That's what makes this a true baseline of
93:            reasons.append(
100:            reasons.append(
108:                reasons.append(
110:                    f"over last {DRIFT_WINDOW} accepted writes exceeds "
136:            # Unknown/alarm register — allow passthrough of writes (e.g. resets)
139:                    self.plant_client.write_register(address, requested, slave=1)
147:            log.warning("BLOCKED write to reg %s: %s", address, reason_text)
148:            log_event(reason_text)
150:            # do not touch the local mirror, do not append to drift history.
158:                self.plant_client.write_register(address, requested, slave=1)
164:            self.temp_accepted_history.append(requested)
166:            self.pressure_accepted_history.append(requested)
168:            self.flow_accepted_history.append(requested)
171:def sync_safe_reads(store, plant_client, lock, stop_event):
177:    while guaranteeing an attacker who writes directly to the plant's own
181:    while not stop_event.is_set():
192:                        store.temp_accepted_history.append(t)
193:                        store.pressure_accepted_history.append(p)
194:                        store.flow_accepted_history.append(f)
208:    stop_event = threading.Event()
211:        args=(store, plant_client, store.lock, stop_event),
223:        stop_event.set()

=== Matching files ===
```

normal-dashboard.png



Observed state: Alarm / zero readings
Electrolyzer temperature: 0.0 °C
Tank pressure: 0.0 bar
H_2 flow rate: 0.0 Nm^3/h
Dashboard status: "GUARDED — RULE-BASED PROTECTION ACTIVE"
Detection log: "Waiting for attacks..."
Interpretation: the dashboard is reachable, but valid live process readings and guard event activity have not been confirmed. The zero values may indicate a communication or polling problem; they do not prove that the plant's actual simulated values are zero.
Process details

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ echo "=== Process details ==="
ps -p 32631,32881 -o pid,ppid,lstart,args

echo
echo "=== Process working directories ==="
for p in 32631 32881 31395; do
    if [ -e "/proc/$p/cwd" ]; then
        printf 'PID %s: ' "$p"
        readlink "/proc/$p/cwd"
    fi
done

echo
echo "=== Guard startup code ==="
nl -ba "$SRC/plant_sim/guard.py" | sed -n '195,235p'

echo
echo "=== Guard connection and dashboard polling code ==="
nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '1,55p'
nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '175,225p'
=== Process details ===
    PID    PPID                  STARTED COMMAND
  32631    5166 Wed Oct 14 07:42:35 2026 python3 plant.py
  32881    4477 Wed Oct 14 07:55:35 2026 python3 plant.py

=== Process working directories ===
PID 32631: /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim
PID 32881: /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim
PID 31395: /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim

=== Guard startup code ===
   195             except Exception as e:
   196                 log.error("Sync error: %s", e)
   197         time.sleep(1.0)
   198
   199
   200 def main():
   201     plant_client = ModbusTcpClient(PLANT_HOST, port=PLANT_PORT)
   202     plant_client.connect()
   203
   204     block = ModbusSequentialDataBlock(0, [670, 340, 900, 0] + [0] * 96)
   205     store = GuardedContext(plant_client, hr=block, zero_mode=True)
   206     context = ModbusServerContext(slaves=store, single=True)
   207
   208     stop_event = threading.Event()
   209     sync_thread = threading.Thread(
   210         target=sync_safe_reads,
   211         args=(store, plant_client, store.lock, stop_event),
   212         daemon=True,
   213     )
   214     sync_thread.start()
   215
   216     log.info("Guard listening on %s:%s — protecting plant at %s:%s",
   217              HOST, GUARD_PORT, PLANT_HOST, PLANT_PORT)
   218     try:
   219         StartTcpServer(context=context, address=(HOST, GUARD_PORT))
   220     except KeyboardInterrupt:
   221         pass
   222     finally:
   223         stop_event.set()
   224         plant_client.close()
   225
   226
   227 if __name__ == "__main__":
   228     main()
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ ps -p 32631,32881 -o pid,ppid,lstart,args
    PID    PPID                  STARTED COMMAND
  32631    5166 Wed Oct 14 07:42:35 2026 python3 plant.py
  32881    4477 Wed Oct 14 07:55:35 2026 python3 plant.py
```

Process working directories

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ for p in 32631 32881 31395; do
  if [ -e "/proc/$p/cwd" ]; then
    printf 'PID %s: ' "$p"
    readlink "/proc/$p/cwd"
  fi
done
PID 32631: /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim
PID 32881: /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim
PID 31395: /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim
```

Guard startup

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/guard.py" | sed -n '195,235p'
   195             except Exception as e:
   196                 log.error("Sync error: %s", e)
   197         time.sleep(1.0)
   198
   199
   200 def main():
   201     plant_client = ModbusTcpClient(PLANT_HOST, port=PLANT_PORT)
   202     plant_client.connect()
   203
   204     block = ModbusSequentialDataBlock(0, [670, 340, 900, 0] + [0] * 96)
   205     store = GuardedContext(plant_client, hr=block, zero_mode=True)
   206     context = ModbusServerContext(slaves=store, single=True)
   207
   208     stop_event = threading.Event()
   209     sync_thread = threading.Thread(
   210         target=sync_safe_reads,
   211         args=(store, plant_client, store.lock, stop_event),
   212         daemon=True,
   213     )
   214     sync_thread.start()
   215
   216     log.info("Guard listening on %s:%s — protecting plant at %s:%s",
   217              HOST, GUARD_PORT, PLANT_HOST, PLANT_PORT)
   218     try:
   219         StartTcpServer(context=context, address=(HOST, GUARD_PORT))
   220     except KeyboardInterrupt:
   221         pass
   222     finally:
   223         stop_event.set()
   224         plant_client.close()
   225
   226
   227 if __name__ == "__main__":
   228     main()
```

dashboard-connection

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '1,55
p'
     1  from flask import Flask, jsonify, render_template_string
     2  from pymodbus.client import ModbusTcpClient
     3  import threading
     4  import time
     5  import json
     6  import os
     7
     8  TARGET_HOST = "127.0.0.1"
     9  TARGET_PORT = 5021
    10  EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
    11
    12  PAGE = """
    13  <!doctype html>
    14  <html>
    15  <head>
    16  <meta charset="utf-8">
    17  <title>H2 Facility — GUARDED</title>
    18  <style>
    19    * { box-sizing: border-box; margin: 0; padding: 0; }
    20    body { font-family: Arial, sans-serif; background: #f4f5f7; color: #1a1a2e; padding: 20px; }
    21    .top-bar { display: flex; justify-content: space-between; align-items: center; background: #1e7e34; color: white; padding: 12px 20px; border-radius: 6px; margin-bottom: 16px; }
    22    .top-bar h1 { font-size: 15px; font-weight: bold; letter-spacing: 1px; }
    23    .top-bar .sub { font-size: 11px; color: #c8e6c9; margin-top: 2px; }
    24    .badge { padding: 6px 16px; border-radius: 4px; font-size: 12px; font-weight: bold; }
    25    .badge-ok { background: white; color: #1e7e34; }
    26    .badge-alarm { background: #c0392b; color: white; }
    27    .grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; margin-bottom: 16px; }
    28    .card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; }
    29    .card-label { font-size: 10px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 8px; }
    30    .card-value { font-size: 36px; font-weight: bold; color: #1a1a2e; font-family: monospace; }
    31    .card-unit { font-size: 13px; color: #6b7280; margin-left: 4px; }
    32    .card-bar-bg { height: 5px; background: #e5e7eb; border-radius: 3px; margin-top: 10px; }
    33    .card-bar { height: 5px; border-radius: 3px; transition: width 0.5s, background 0.5s; }
    34    .bar-ok { background: #1e7e34; }
    35    .bar-alarm { background: #c0392b; }
    36    .card-range { display: flex; justify-content: space-between; font-size: 10px; color: #9ca3af; margin-top: 3px; }
    37    .plant-card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; margin-bottom: 16px; }
    38    .plant-card h3 { font-size: 11px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 12px; }
    39    .log-card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; margin-top: 16px; }
```

dashboard-polling

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '175,225p'
   175
   176  async function poll(){
   177    try{
   178      const d=await(await fetch('/api/state')).json();
   179      const temp=d.temp/10, pres=d.pressure/10, flow=d.flow/10;
   180      document.getElementById('v-temp').textContent=temp.toFixed(1);
   181      document.getElementById('v-pres').textContent=pres.toFixed(1);
   182      document.getElementById('v-flow').textContent=flow.toFixed(1);
   183      document.getElementById('diag-temp').textContent=temp.toFixed(1)+' °C';
   184      document.getElementById('diag-pres').textContent=pres.toFixed(1)+' bar';
   185      document.getElementById('diag-flow').textContent=flow.toFixed(1)+' Nm³/h';
   186      document.getElementById('bar-temp').style.width=clamp((temp-55)/25,0,1)*100+'%';
   187      document.getElementById('bar-pres').style.width=clamp((pres-25)/30,0,1)*100+'%';
   188      document.getElementById('bar-flow').style.width=clamp((flow-40)/130,0,1)*100+'%';
   189
   190      const fillFrac = clamp(pres/100, 0, 1);
   191      const maxH = 75, bottomY = 90;
   192      const h = maxH * fillFrac;
   193      document.getElementById('tank-inner').setAttribute('y', bottomY - h);
   194      document.getElementById('tank-inner').setAttribute('height', h);
   195
   196      const alarm = pres > 40 || pres < 28;
   197      document.getElementById('bar-pres').className = 'card-bar ' + (alarm ? 'bar-alarm' : 'bar-ok');
   198      setAlarm(alarm);
   199      document.getElementById('footer').textContent={d.connected?'Live — GUARD ACTIVE':'Disconnected'}+' — '+new Date().toTimeString().slice(0,8);
   200    }catch(e){document.getElementById('footer').textContent='Error';}
   201
   202    try {
   203      const events = await (await fetch('/api/events')).json();
   204      if (events.length !== lastEvents) {
   205        lastEvents = events.length;
   206        const scroll = document.getElementById('log-scroll');
   207        if (events.length === 0) {
   208          scroll.innerHTML = '<div class="log-row"><span class="log-ts">--:--:--</span><span class="log-ts">No events yet.</span></div>';
   209        } else {
   210          scroll.innerHTML = events.slice().reverse().map(e =>
   211            `<div class="log-row"><span class="log-ts">${(e.timestamp||'').slice(11,19)}</span><span class="log-tag-block">BLOCKED</span><span class="log-msg">${e.reason||''}</span></div>`
   212          ).join('');
```

guard-configuration-check

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ echo "=== Plant port check ==="
ss -ltnp | grep '127.0.0.1:5020'

echo
echo "=== Guard bind and plant target ==="
nl -ba "$SRC/plant_sim/guard.py" | sed -n '15,50p'

echo
echo "=== Current source and evidence paths ==="
printf 'SRC=%s\nEV=%s\n' "$SRC" "$EV"
=== Plant port check ===
LISTEN 0      100        127.0.0.1:5020      0.0.0.0:*    users:(("python3",pid=32631,fd=6))

=== Guard bind and plant target ===
    15
    16  logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
    17  log = logging.getLogger("guard")
    18
    19  PLANT_HOST = "127.0.0.1"
    20  PLANT_PORT = 5020
    21  HOST = "127.0.0.1"
    22  GUARD_PORT = 5021
    23
    24  REG_TEMP = 0
    25  REG_PRESSURE = 1
    26  REG_FLOW = 2
    27  REG_ALARM = 3
    28
    29  TEMP_RANGE     = (600, 750)   # tenths of a degree C
    30  PRESSURE_RANGE = (300, 400)   # tenths of a bar
    31  FLOW_RANGE     = (500, 1500)  # tenths of Nm3/h
    32
    33  MAX_TEMP_STEP      = 30
    34  MAX_PRESSURE_STEP  = 50
    35  MAX_FLOW_STEP      = 100
    36
    37  # --- Rolling-baseline drift detection ---
    38  # Instead of comparing write N to write N-1 (which a slow-ramp attack can
    39  # always stay under), we compare the CURRENT accepted value to the accepted
    40  # value from DRIFT_WINDOW writes ago. This catches cumulative creep even
```

guard-decision-rules

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/guard.py" | sed -n '60,175p'
    60
    61  class GuardedContext(ModbusSlaveContext):
    62      """
    63      A Modbus write-through firewall.
    64
    65      Design rule: this class NEVER accepts a value it hasn't itself validated.
    66      - Absolute range check: hard reject outside physical safe limits.
    67      - Step check: hard reject any single write that moves too far too fast.
    68      - Rolling-baseline drift check: compares the CURRENT accepted value to
    69        the accepted value from N writes ago, so an attacker cannot bypass
    70        detection by taking many small steps.
    71
    72      On any rejection: the write is dropped entirely. Nothing is forwarded
    73      to the plant, nothing is written to the local mirror. The guard's
    74      exposed state (what the guarded dashboard sees) simply does not move.
    75      """
    76
    77      def __init__(self, plant_client, *args, **kwargs):
    78          super().__init__(*args, **kwargs)
    79          self.plant_client = plant_client
    80          self.lock = threading.Lock()
    81          # Rolling baseline histories store ACCEPTED values only, not
    82          # attempted writes. That's what makes this a true baseline of
    83          # "what the plant has actually been allowed to do."
    84          self.temp_accepted_history     = deque(maxlen=DRIFT_WINDOW)
    85          self.pressure_accepted_history = deque(maxlen=DRIFT_WINDOW)
    86          self.flow_accepted_history     = deque(maxlen=DRIFT_WINDOW)
    87
    88      def _check(self, address, requested, current, value_range, max_step,
    89                 drift_history, drift_threshold, unit_label, unit_divisor):
    90          reasons = []
    91
    92          if not (value_range[0] <= requested <= value_range[1]):
    93              reasons.append(
    94                  f"{requested/unit_divisor:.1f}{unit_label} outside safe range "
    95                  f"{value_range[0]/unit_divisor:.1f}-{value_range[1]/unit_divisor:.1f}{unit_label}"
    96              )
    97
    98          step = abs(requested - current)
    99          if step > max_step:
```

```
        if step > max_step:
            reasons.append(
                f"step of {step/unit_divisor:.1f}{unit_label} exceeds max "
                f"{max_step/unit_divisor:.1f}{unit_label}"
            )

        if len(drift_history) == DRIFT_WINDOW:
            drift = requested - drift_history[0]
            if abs(drift) > drift_threshold:
                reasons.append(
                    f"cumulative drift of {drift/unit_divisor:.1f}{unit_label} "
                    f"over last {DRIFT_WINDOW} accepted writes exceeds "
                    f"{drift_threshold/unit_divisor:.1f}{unit_label} limit"
                )

        return reasons

    def setValues(self, fc_as_hex, address, values):
        requested = values[0]
        reasons = []

        if address == REG_TEMP:
            current = self.getValues(3, REG_TEMP, count=1)[0]
            reasons = self._check(address, requested, current, TEMP_RANGE,
                                  MAX_TEMP_STEP, self.temp_accepted_history,
                                  DRIFT_THRESHOLD_TEMP, "C", 10)
        elif address == REG_PRESSURE:
            current = self.getValues(3, REG_PRESSURE, count=1)[0]
            reasons = self._check(address, requested, current, PRESSURE_RANGE,
                                  MAX_PRESSURE_STEP, self.pressure_accepted_history,
                                  DRIFT_THRESHOLD_PRESSURE, " bar", 10)
        elif address == REG_FLOW:
            current = self.getValues(3, REG_FLOW, count=1)[0]
            reasons = self._check(address, requested, current, FLOW_RANGE,
                                  MAX_FLOW_STEP, self.flow_accepted_history,
                                  DRIFT_THRESHOLD_FLOW, " Nm3/h", 10)
        else:
            # Unknown/alarm register — allow passthrough of writes (e.g. resets)
            with self.lock:
                try:
                    self.plant_client.write_register(address, requested, slave=1)
                except Exception as e:
                    self.plant_client.write_register(address, requested, slave=1)
                except Exception as e:
                    log.error("Forward failed: %s", e)
            super().setValues(fc_as_hex, address, values)
            return

        if reasons:
            reason_text = "; ".join(reasons)
            log.warning("BLOCKED write to reg %s: %s", address, reason_text)
            log_event(reason_text)
            # Option A behavior: reject completely. Do not touch the plant,
            # do not touch the local mirror, do not append to drift history.
            # The last known-safe accepted value remains authoritative.
            return

        # Accepted — forward to the real plant AND update local mirror AND
        # record it in the rolling baseline history.
        with self.lock:
            try:
                self.plant_client.write_register(address, requested, slave=1)
            except Exception as e:
                log.error("Forward failed: %s", e)
        super().setValues(fc_as_hex, address, values)

        if address == REG_TEMP:
            self.temp_accepted_history.append(requested)
        elif address == REG_PRESSURE:
            self.pressure_accepted_history.append(requested)
        elif address == REG_FLOW:
            self.flow_accepted_history.append(requested)


def sync_safe_reads(store, plant_client, lock, stop_event):
    """
    Periodically reads the real plant's telemetry and mirrors it into the
    guard's exposed state — but ONLY if every value read is within the
    absolute safe range. This keeps normal plant drift (temperature
```

source-based decision table

Check	Input / condition	Guard action
Absolute safe range	Temperature: 60–75°C	Reject if outside range
Absolute safe range	Pressure: 30–40 bar	Reject if outside range
Absolute safe range	Flow: 50–150 Nm³/h	Reject if outside range
Single-write step	Temperature change greater than 3°C	Reject
Single-write step	Pressure change greater than 5 bar	Reject
Single-write step	Flow change greater than 10 Nm³/h	Reject
Rolling-baseline drift	After 3 accepted writes, requested value differs from the oldest retained accepted value by more than the configured threshold	Reject
Other registers	Register is not temperature, pressure, or flow	Forward write and update local mirror; errors are logged
Accepted	All applicable checks pass	Forward to plant, update

process write		mirror, add to accepted history
Rejected process write	One or more checks fail	Log warning and event; do not forward or update mirror/history

The drift thresholds are 2°C, 2 bar, and 4 Nm³/h, respectively. The source uses strict “greater than” comparisons, so a value exactly at a threshold does not trigger that particular check. Source references: guard.py lines 29-45 and 88-168.

Important limitation: The other-register path allows writes through without these three process-value checks. Do not describe it as universally blocking unsafe writes.

safe-read-and-startup

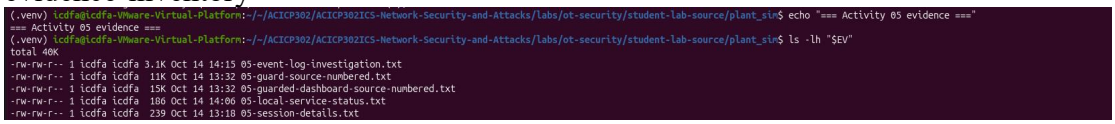
```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/guard.py" | sed -n '171,235p'
   171  def sync_safe_reads(store, plant_client, lock, stop_event):
   172      """
   173      Periodically reads the real plant's telemetry and mirrors it into the
   174      guard's exposed state — but ONLY if every value read is within the
   175      absolute safe range. This keeps normal plant drift (temperature
   176      wandering a bit under real physics) visible on the guarded dashboard,
   177      while guaranteeing an attacker who writes directly to the plant's own
   178      port (bypassing the guard entirely) can never have those corrupted
   179      values bleed into the guard's mirror.
   180      """
   181      while not stop_event.is_set():
   182          with lock:
   183              try:
   184                  result = plant_client.read_holding_registers(0, 4, slave=1)
   185                  if not result.isError():
   186                      regs = result.registers
   187                      t, p, f = regs[REG_TEMP], regs[REG_PRESSURE], regs[REG_FLOW]
   188                      if (TEMP_RANGE[0]     <= t <= TEMP_RANGE[1] and
   189                          PRESSURE_RANGE[0] <= p <= PRESSURE_RANGE[1] and
   190                          FLOW_RANGE[0]     <= f <= FLOW_RANGE[1]):
   191                          ModbusSlaveContext.setValues(store, 3, 0, list(regs))
   192                          store.temp_accepted_history.append(t)
   193                          store.pressure_accepted_history.append(p)
   194                          store.flow_accepted_history.append(f)
   195              except Exception as e:
   196                  log.error("Sync error: %s", e)
   197          time.sleep(1.0)
   198
   199
   200  def main():
   201      plant_client = ModbusTcpClient(PLANT_HOST, port=PLANT_PORT)
   202      plant_client.connect()
   203
   204      block = ModbusSequentialDataBlock(0, [670, 340, 900, 0] + [0] * 96)
   205      store = GuardedContext(plant_client, hr=block, zero_mode=True)
   206      context = ModbusServerContext(slaves=store, single=True)
   207
   208      stop_event = threading.Event()
   209      sync_thread = threading.Thread(
```

guarded-dashboard-source

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ nl -ba "$SRC/plant_sim/dashboard_guarded.py" | sed -n '1,180p'
     1  from flask import Flask, jsonify, render_template_string
     2  from pymodbus.client import ModbusTcpClient
     3  import threading
     4  import time
     5  import json
     6  import os
     7
     8  TARGET_HOST = "127.0.0.1"
     9  TARGET_PORT = 5021
    10  EVENT_LOG_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "events.jsonl")
    11
    12  PAGE = """
    13  <!doctype html>
    14  <html>
    15  <head>
    16  <meta charset="utf-8">
    17  <title>H2 Facility — GUARDED</title>
    18  <style>
    19    * { box-sizing: border-box; margin: 0; padding: 0; }
    20    body { font-family: Arial, sans-serif; background: #f4f5f7; color: #1a1a2e; padding: 20px; }
    21    .top-bar { display: flex; justify-content: space-between; align-items: center; background: #1e7e34; color: white; padding: 12px 20px; border-radius: 6px; margin-bottom: 16px; }
    22    .top-bar h1 { font-size: 15px; font-weight: bold; letter-spacing: 1px; }
    23    .top-bar .sub { font-size: 11px; color: #c8e6c9; margin-top: 2px; }
    24    .badge { padding: 6px 16px; border-radius: 4px; font-size: 12px; font-weight: bold; }
    25    .badge-ok { background: white; color: #1e7e34; }
    26    .badge-alarm { background: #c0392b; color: white; }
    27    .grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 12px; margin-bottom: 16px; }
    28    .card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; }
    29    .card-label { font-size: 10px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 8px; }
    30    .card-value { font-size: 36px; font-weight: bold; color: #1a1a2e; font-family: monospace; }
    31    .card-unit { font-size: 13px; color: #6b7280; margin-left: 4px; }
    32    .card-bar-bg { height: 5px; background: #e5e7eb; border-radius: 3px; margin-top: 10px; }
    33    .card-bar { height: 5px; border-radius: 3px; transition: width 0.5s, background 0.5s; }
    34    .bar-ok { background: #1e7e34; }
    35    .bar-alarm { background: #c0392b; }
    36    .card-range { display: flex; justify-content: space-between; font-size: 10px; color: #9ca3af; margin-top: 3px; }
    37    .plant-card { background: white; border: 1px solid #dde1e7; border-radius: 6px; padding: 16px; margin-bottom: 16px; }
    38    .plant-card h3 { font-size: 11px; text-transform: uppercase; letter-spacing: 1px; color: #6b7280; margin-bottom: 12px; }
```



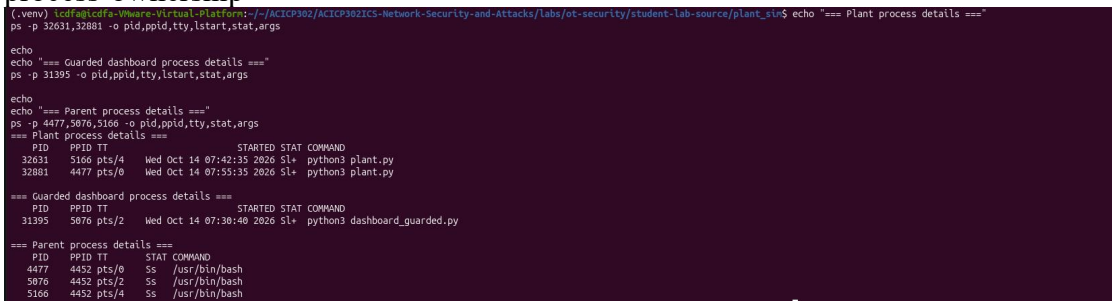
evidence-inventory



service-diagnostic



process-ownership



We have identified the terminal ownership:

Terminal	Process	Status
pts/0	plant.py PID 32881	Running
pts/2	dashboard_guarded.py PID 31395	Running
pts/4	plant.py PID 32631	Running
—	guard.py	Not running
—	dashboard.py	Not running

documented-startup

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ grep -n -A 35 -B 8 'guard.py\|dashboard_guarded.py' \
  "$LAB/STUDENT_LAB_GUIDE.md"
105-cd plant_sim
106-```
107-
108-Start the services in this order, one process per terminal:
109-
110-| Terminal | Command | Expected behaviour |
111-|---|---|---|
112- | 1 | `python3 plant.py` | Simulated process starts on loopback port 5020. |
113- | 2 | `python3 guard.py` | Guard starts on loopback port 5021 and forwards accepted writes to the local plant. |
114- | 3 | `python3 dashboard.py` | Direct dashboard starts on loopback port 8080. |
115:| 4 | `python3 dashboard_guarded.py` | Guarded dashboard starts on loopback port 8081. |
116-
117-Open the dashboards in Ubuntu:
118-
119:- Direct view: <http://127.0.0.1:8080>
120:- Guarded view: <http://127.0.0.1:8081>
121-
122-Initial values are approximately 67.0 °C, 34.0 bar and 90.0 Nm³/h; values may move slightly as the simulator runs. The guard is rule-based, not an AI model.
123-
124-## 7. Baseline and packet capture
125-
126-Before any demonstration, record the VM name, date/time, source commit, loopback check, disconnected network adapter, dashboard readings and normal observations. Save learner-created screenshots with relevant context.
127-
128-For the packet activity, capture only a short period of the Ubuntu guest's loopback (`lo`) traffic for the documented Modbus/TCP lab port. Do not capture other interfaces or unrelated traffic. In Wire shark, identify one request and matching response, then relate the observed fields to your register map and process state. Stop the capture and store it only in the instructor-approved private evidence lo cation.
129-
130-## 8. Run approved demonstrations
131-
132-Keep the four services running and use a fifth Ubuntu terminal. Activate the same environment and enter the source folder:
133-
134-```bash
135-cd ~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source
136-source .venv/bin/activate
137-cd plant_sim
```

```
133-```bash
134-cd ~/course/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source
135-source .venv/bin/activate
136-cd plant_sim
137-```
138-
139-
140-Use the activity test plan and instructor approval. Select at least three permitted demonstrations; run them one at a time and use the supplied arguments unchanged. Each script is limited to the local simulator; port `5020` is the direct path and port `5021` is the guarded path.
141-
142-| Script | Demonstration | Supplied command pair |
143-|---|---|---|
144- | `attack_hold.py` | Requests a pressure value outside the simulated range | `python3 attack_hold.py 5020` then `python3 attack_hold.py 5021` |
145- | `false_data_attack.py` | Sends the provided batches of out-of-range process values | `python3 false_data_attack.py 5020` then `python3 false_data_attack.py 5021` |
146- | `slow_drift_attack.py` | Sends the provided incremental pressure changes | `python3 slow_drift_attack.py 5020` then `python3 slow_drift_attack.py 5021` |
147- | `replay_attack.py` | Repeats the provided pressure write | `python3 replay_attack.py 5020` then `python3 replay_attack.py 5021` |
148- | `recon_scan.py` | Reads the limited register range documented by the script | `python3 recon_scan.py 5020` then `python3 recon_scan.py 5021` |
149- | `dos_attack.py` | Fixed demonstration of 200 read requests using 10 threads | Run only if the instructor approves; then use `python3 dos_attack.py 5020` followed by `python3 dos_attack.py 5021` |
150-
```

prestart-check

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ echo "=== Confirm lab source directory ==="
pwd

echo
echo "=== Confirm virtual environment ==="
python3 -c 'import sys; print(sys.executable)'

echo
echo "=== Confirm required source files ==="
ls -l "$SRC/plant_sim/plant.py" \
      "$SRC/plant_sim/guard.py" \
      "$SRC/plant_sim/dashboard.py" \
      "$SRC/plant_sim/dashboard_guarded.py"

echo
echo "=== Confirm guard and plant ports ==="
ss -ltnp | grep -E '127\.0\.0\.1:(5020|5021|8080|8081)' || true
=== Confirm lab source directory ===
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim

=== Confirm virtual environment ===
/home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/.venv/bin/python3

=== Confirm required source files ===
-rw-rw-r-- 1 icdfa icdfa 11683 Oct 13 15:46 /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard.py
-rw-rw-r-- 1 icdfa icdfa 13161 Oct 13 15:46 /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/dashboard_guarded.py
-rw-rw-r-- 1 icdfa icdfa  9198 Oct 13 15:46 /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/guard.py
-rw-rw-r-- 1 icdfa icdfa  3270 Oct 13 15:46 /home/icdfa/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim/plant.py

=== Confirm guard and plant ports ===
LISTEN 0      128        127.0.0.1:8081       0.0.0.0:*    users:(("python3",pid=31395,fd=4))
LISTEN 0      100        127.0.0.1:5020       0.0.0.0:*    users:(("python3",pid=32631,fd=6))
```

duplicate-plant-check

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ echo "=== Plant listener ==="
ss -ltnp | grep '127.0.0.1:5020' || true

echo
echo "=== Remaining plant processes ==="
ps -eo pid,tty,stat,args | grep '[p]lant.py'
=== Plant listener ===
LISTEN 0      100        127.0.0.1:5020       0.0.0.0:*    users:(("python3",pid=32631,fd=6))

=== Remaining plant processes ===
  32631 pts/4    Sl+  python3 plant.py
  32881 pts/0    Sl+  python3 plant.py
```

guard-startup

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source/plant_sim$ python3 guard.py
2026-10-14 16:17:37,931 INFO Guard listening on 127.0.0.1:5021 — protecting plant at 127.0.0.1:5020
2026-10-14 16:17:37,935 INFO Server listening.
```

final-service-status

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source$ ss -ltnp | grep -E '127\.0\.0\.1:(5020|5021|8080|8081)' || true
LISTEN 0      100        127.0.0.1:5021       0.0.0.0:*    users:(("python3",pid=62025,fd=7))
LISTEN 0      100        127.0.0.1:5020       0.0.0.0:*    users:(("python3",pid=32631,fd=6))
```

Summary

Activity 05 involved reviewing the OT security guard and guarded dashboard, examining the configured process-value limits, write-step restrictions, rolling drift thresholds, and event-logging mechanisms. The plant simulator and security guard were confirmed listening on their documented loopback ports, 127.0.0.1:5020 and 127.0.0.1:5021, respectively. However, the guard's rejection behavior and event logging were not fully verified through a controlled runtime test. Therefore, Activity 06 will build on these findings by comparing direct and guarded responses using instructor-approved demonstrations, recording observed outcomes, and verifying

recovery without assuming that the guard successfully blocks every unauthorized or abnormal write.

Activity 06 - Controlled security validation

Objective

Compared direct and guarded communication paths in the authorized local ICS simulator to observe how rule-based protection handles simulated attack scenarios.

2. Environment

Direct path: 127.0.0.1:5020
Guarded path: 127.0.0.1:5021
Dashboard: http://127.0.0.1:8080
Temperature range: 60-75 °C
Pressure range: 30-40 bar
Hydrogen flow range: 50–150 Nm^3/h

3. Scenarios completed

Attack Hold: The direct path reached 95.0 bar, with all 76 readings out of range. The guarded path's peak was 31.7 bar, with zero out-of-range readings.

False Data Injection: The direct path reported unsafe temperature, pressure, and flow values. The guarded path kept all six reported readings for each measurement within the configured ranges.

Slow Drift: Actual pressure readings remained at or below 40 bar on both paths. The direct path reached 40.0 bar; the guarded path's highest reported actual pressure was 39.3 bar.

4. Recovery and analysis

The final displayed dashboard readings were within the configured limits. The direct pressure reading was at the 40-bar upper limit. No Slow Drift-specific guard log entry was confirmed, so the report does not claim that a corresponding log event was recorded.

The results demonstrate the usefulness of the configured rules in these particular simulations, but they do not prove protection against every ICS attack or establish that the guard is a real safety system.

Evidence saved

script-review

```
(.venv) icdfa@icdfa-VMware-Virtual-Platform:~/~/ACICP302/ACICP302ICS-Network-Security-and-Attacks/labs/ot-security/student-lab-source$ for f in attack_hold.py false_data_attack.py slow_drift_attack.py; do

    echo
    echo "========== $f =========="
    sed -n '1,220p' "$SRC/plant_sim/$f"
done

========== attack_hold.py ==========
from pymodbus.client import ModbusTcpClient
import time
import sys

HOST = "127.0.0.1"
PORT = int(sys.argv[1]) if len(sys.argv) > 1 else 5020

# Safe range for pressure, in tenths of a bar (matches guard.py / plant.py)
PRESSURE_SAFE = (300, 400)

print(f"--- ATTACK HOLD: targeting port {PORT} ---")
client = ModbusTcpClient(HOST, port=PORT)
client.connect()

readings = []          # (elapsed_seconds, actual_value)
attack_value = 950     # 95.0 bar — well outside safe range
writes_sent = 0

start = time.time()
deadline = start + 8

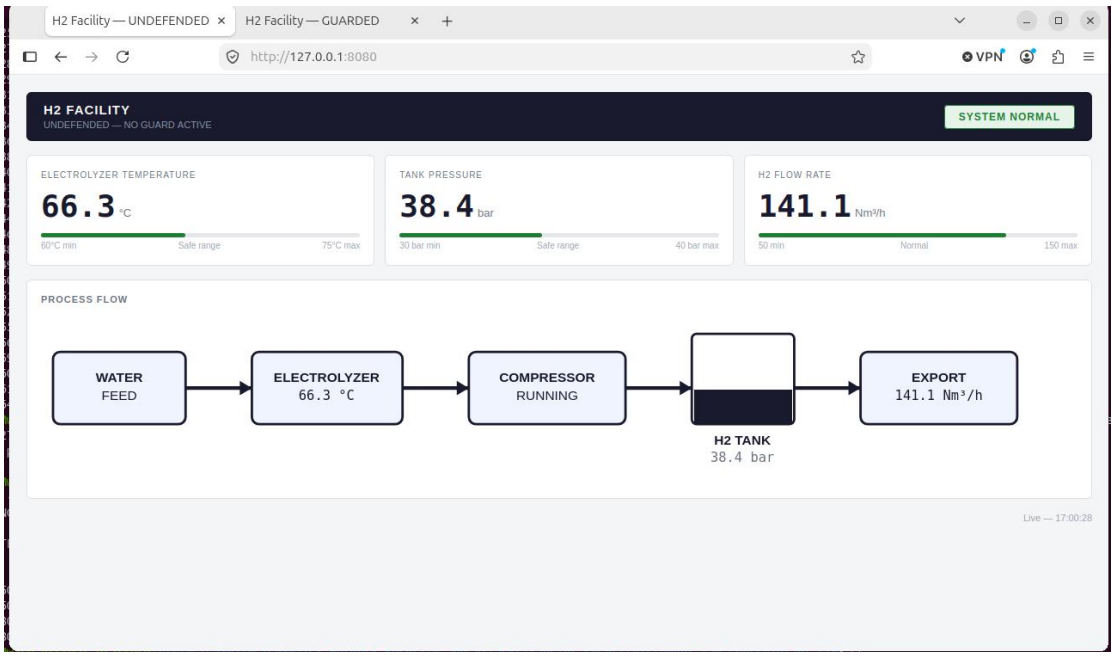
while time.time() < deadline:
    client.write_register(1, attack_value, slave=1)
    writes_sent += 1
    # Read back immediately — this is the actual live register state
    # right after the write, before any self-heal tick can smooth it over.
    r = client.read_holding_registers(1, 1, slave=1)
    if not r.isError():
        elapsed = time.time() - start
        readings.append((elapsed, r.registers[0]))
    time.sleep(0.1)   # poll fast — plant's own tick loop runs every 1s

client.close()
```

Record the script-review observations
The source review shows these candidate scenarios:

Script	Behavior	Key observation
attack_hold.py	Repeated pressure writes for 8 seconds	Requests 95.0 bar, outside the guard's documented 30-40 bar range.
false_data_attack.py	Six batches of process-value writes	Requests temperature 110.0°C, pressure 55.0 bar, and flow 200.0 Nm³/h, all outside the documented ranges.
slow_drift_attack.py	Twelve incremental pressure writes, two seconds apart	Increases the requested pressure by 1.0 bar per step; the guard's rolling drift and step limits may affect the outcome.

baseline-services
Direct: http://127.0.0.1:8080

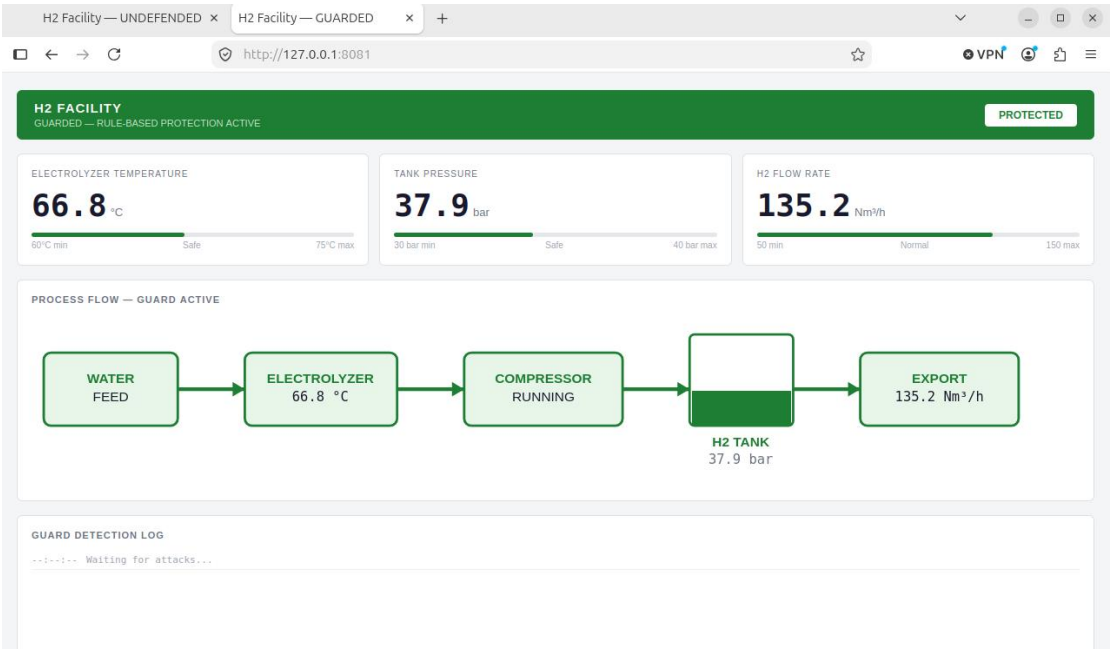


Baseline observations

Process variable	Direct dashboard	Documented range	Status
Electrolyzer temperature	62.9°C	60–75°C	Within range
Tank pressure	35.5 bar	30–40 bar	Within range
Hydrogen flow rate	143.5 Nm³/h	50–150 Nm³/h	Within range
System status	System normal	Normal baseline	Reported normal

The direct dashboard identifies the system as "UNDEFENDED — NO GUARD ACTIVE." These readings establish the initial direct-path baseline; they do not establish how the guarded path will respond.

Guarded: http://127.0.0.1:8081



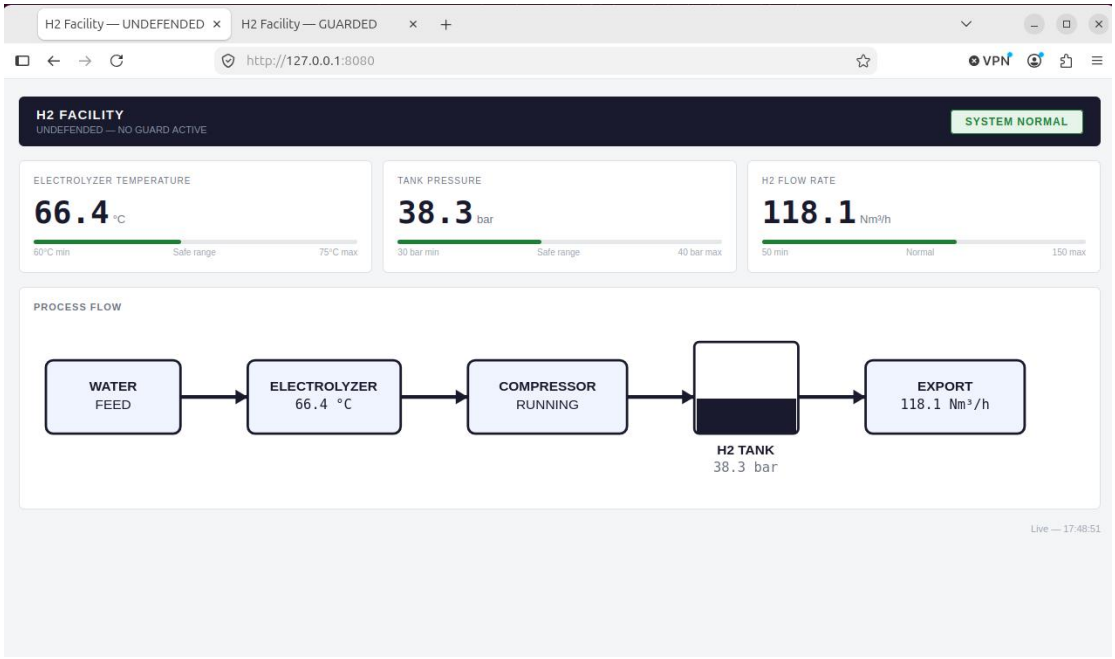
Baseline comparison

Process variable	Direct path	Guarded path	Documented range
Electrolyzer temperature	62.9°C	62.3°C	60-75°C
Tank pressure	35.5 bar	35.4 bar	30-40 bar
Hydrogen flow rate	143.5 Nm³/h	141.0 Nm³/h	50-150 Nm³/h
Dashboard status	System normal	Protected	Normal baseline

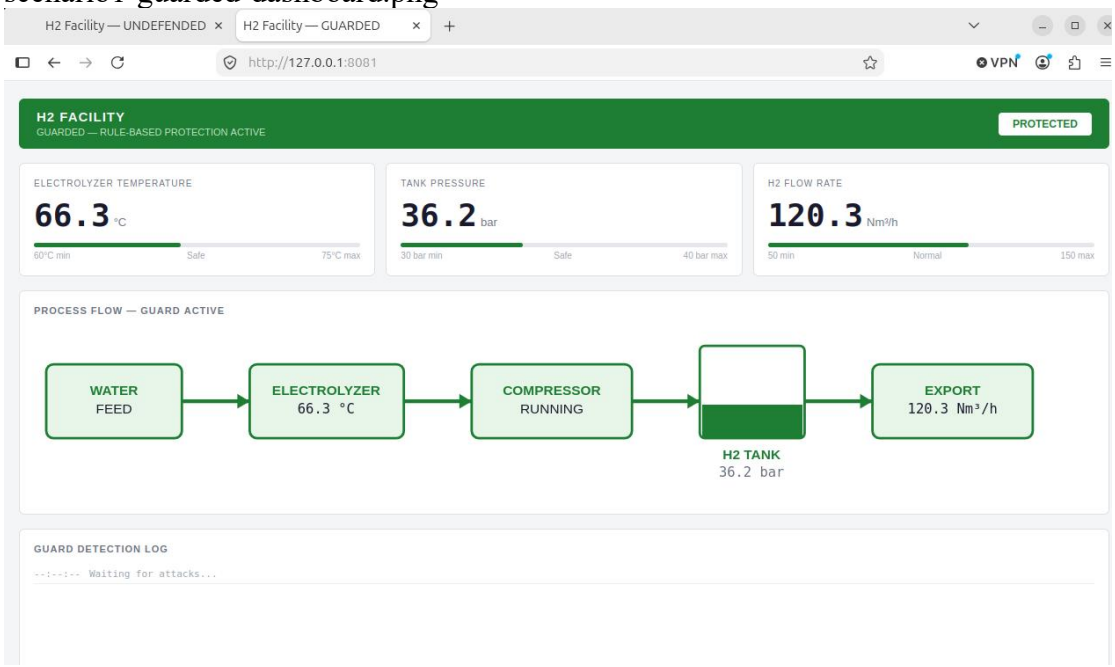
scenario1-precheck



scenario1-direct-dashboard.png



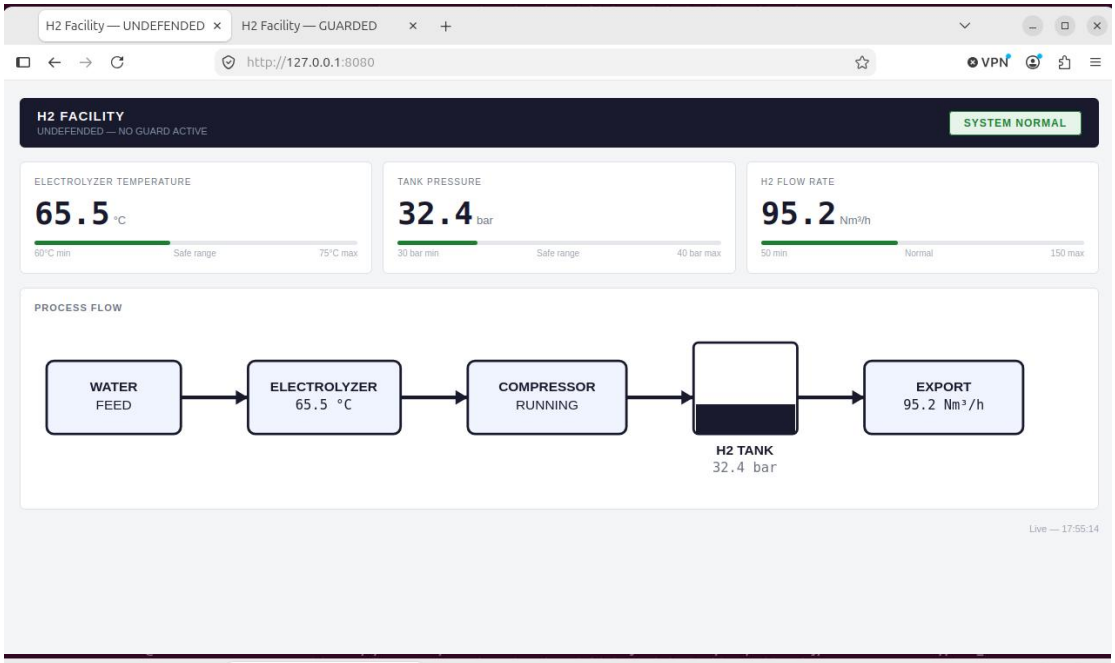
scenario1-guarded-dashboard.png

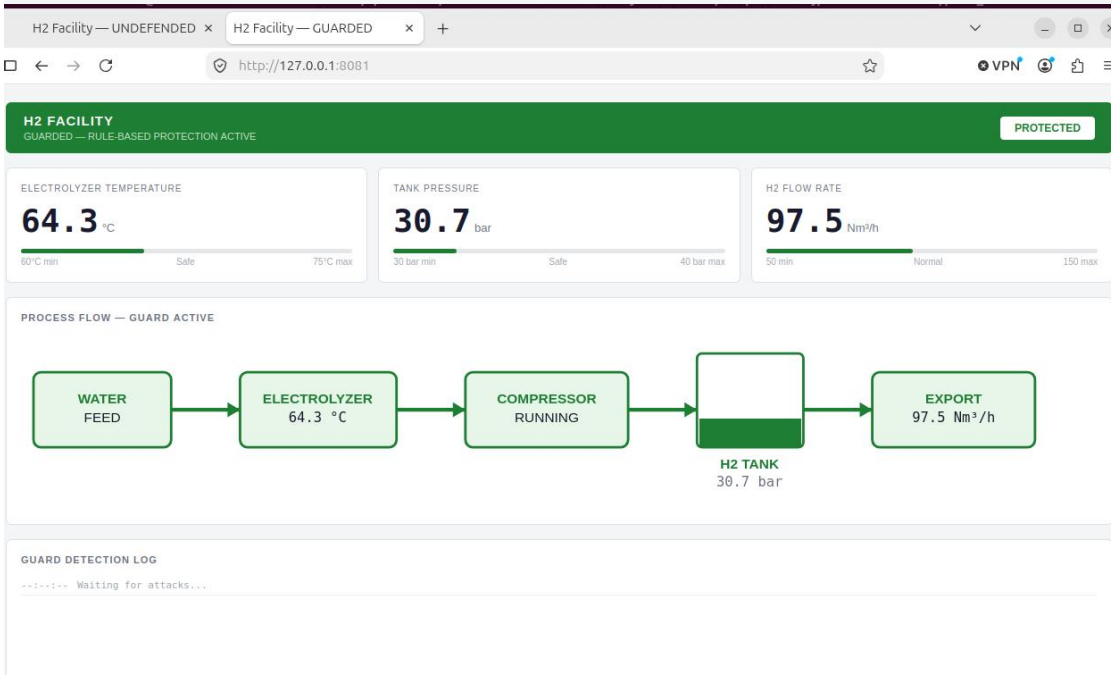


Measurement	Direct dashboard	Guarded dashboard
Temperature	66.5°C	66.1°C
Tank pressure	40.0 bar	39.0 bar
H_2 flow rate	119.8 Nm^3/h	119.7 Nm^3/h
Status	System normal	Protected

The direct pressure is at the stated limit, not above it. Since the readings may have been taken at different times, avoid treating the 1.0-bar difference as proof of guard effectiveness.

scenario1-recovery

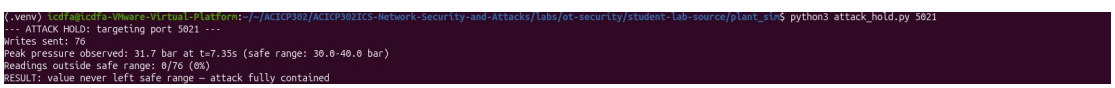


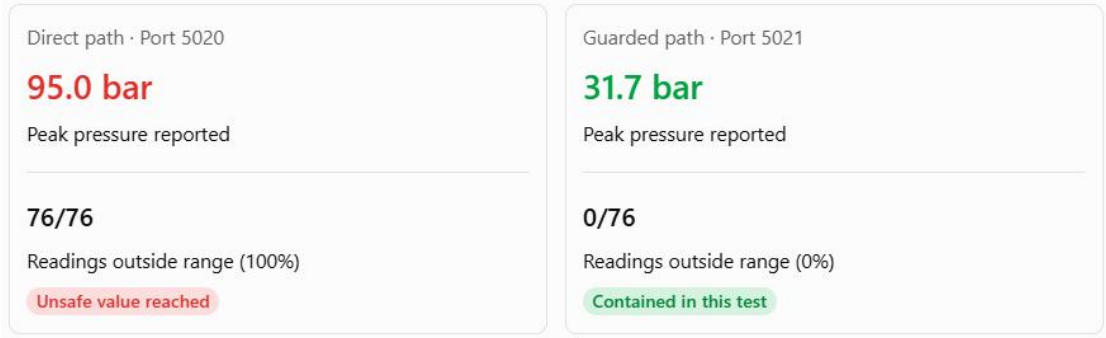


Scenario 1 recovery check

Measurement	Direct path	Guarded path	Allowed range
Electrolyzer temperature	65.6°C	66.3°C	60–75°C
Tank pressure	35.4 bar	34.7 bar	30–40 bar
H_2 flow rate	96.4 Nm^3/h	95.4 Nm^3/h	50–150 Nm^3/h
Dashboard status	System normal	Protected	—

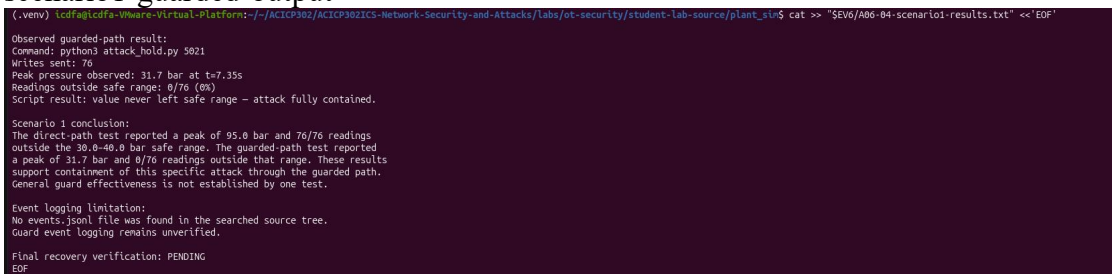
Recovery assessment: The dashboard values are within the stated operating ranges. This supports recovery of the displayed process values, although the event log and service status have not yet been verified.



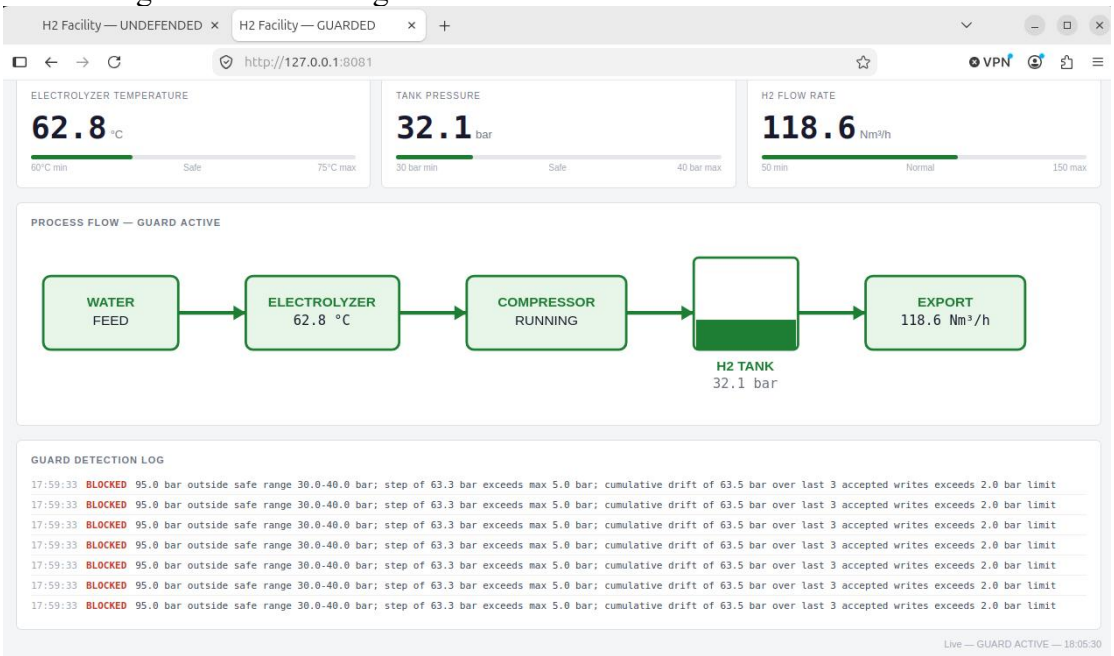


The direct-path script reported an unsafe value of 95.0 bar, while the guarded-path script reported a peak of 31.7 bar and no readings outside the 30-40 bar range. This supports the conclusion that the guard contained the tested writes under these conditions. It does not establish that every possible unsafe write would be blocked.

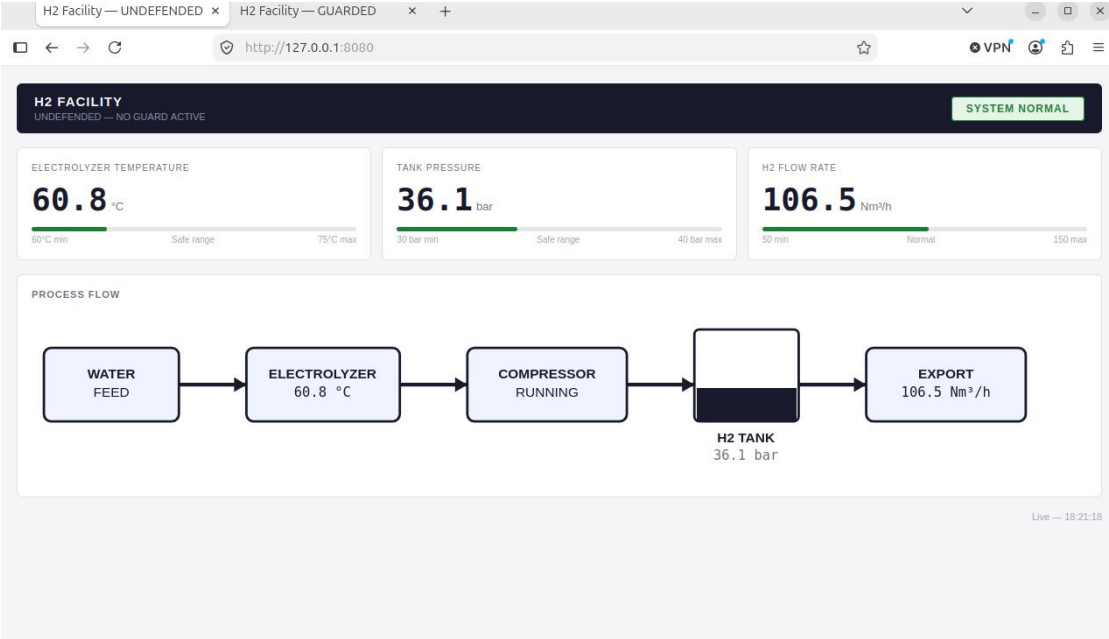
scenario1-guarded-output



scenario1-guard-detection-log

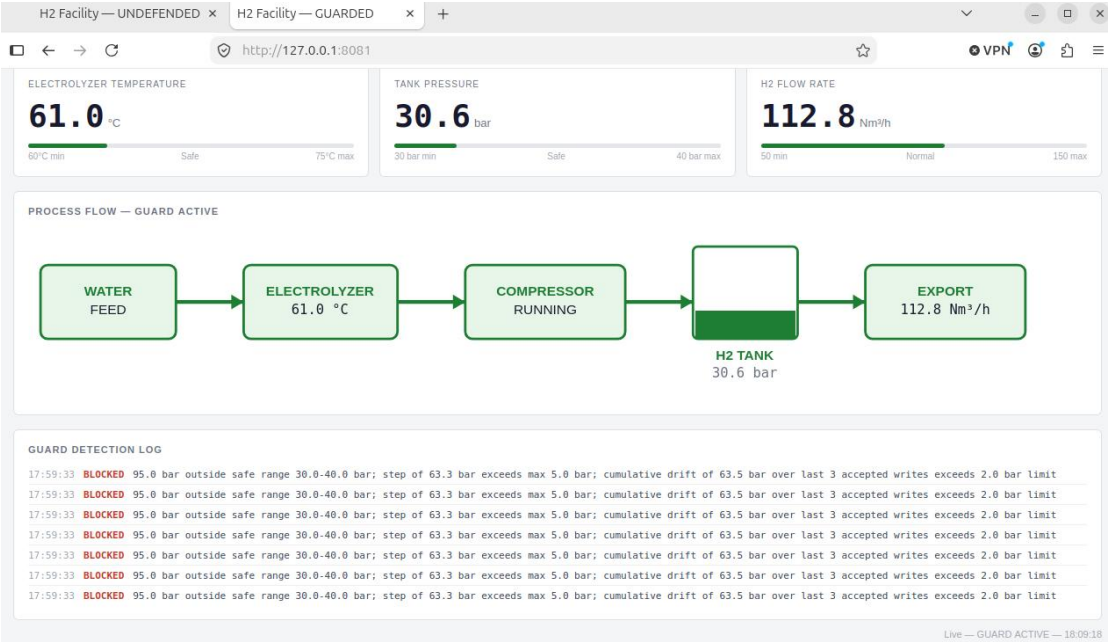


scenario1-direct-recovery



Measurement	Direct path	Guarded path
Temperature	60.0°C	63.2°C
Tank pressure	34.7 bar	30.6 bar
H_2 flow rate	104.4 Nm³/h	120.1 Nm³/h
Dashboard status	System normal	Protected

Assessment: Both dashboards currently display values within the specified ranges and are responsive. This supports recovery of the displayed process values following Scenario 1, although it does not independently verify every internal simulator state.



Recovery and detection evidence

Measurement	Current guarded reading	Permitted range	Assessment
Electrolyzer temperature	63.2°C	60–75°C	Within range

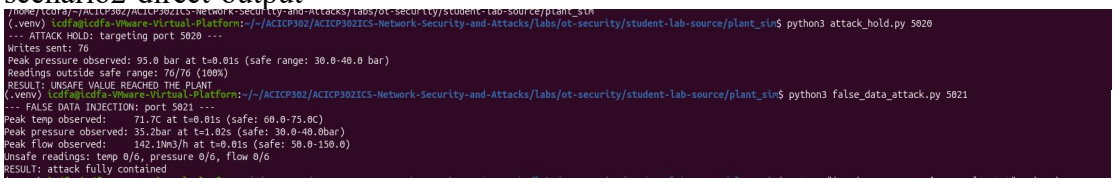
Tank pressure	30.6 bar	30–40 bar	Within range
H_2 flow rate	120.1 Nm^3/h	50–150 Nm^3/h	Within range

The pressure is close to the lower limit of 30 bar, but it remains within the stated range. Continue to monitor the dashboard rather than assuming the plant is fully stable from one reading.

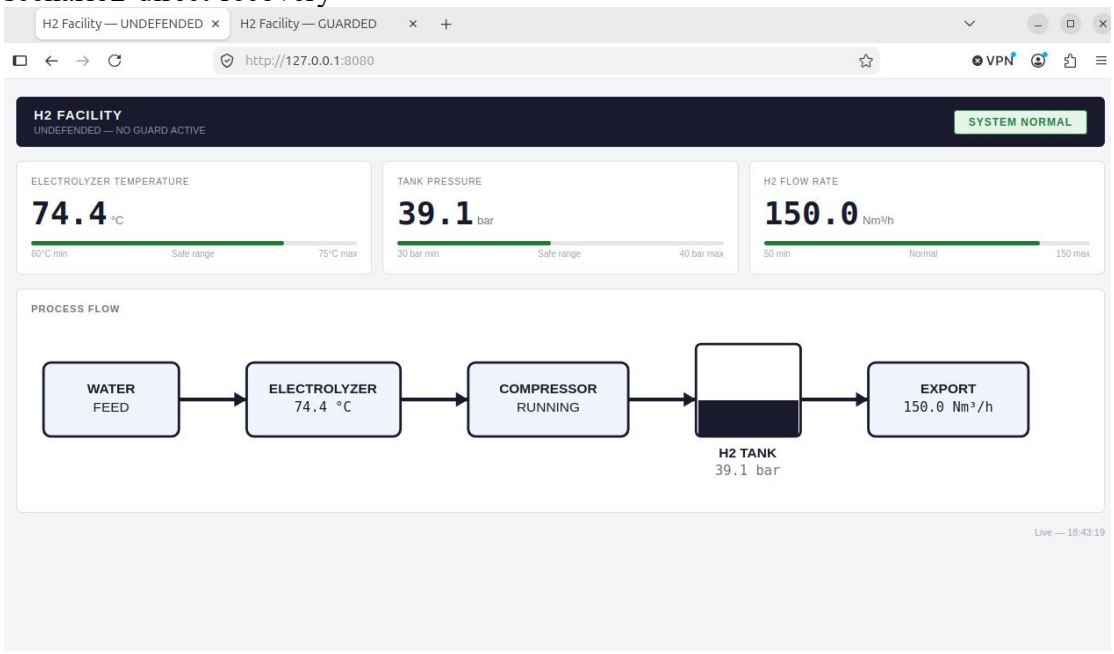
The detection log records attempts to write 95.0 bar and identifies violations of the absolute pressure limit, maximum step size, and cumulative drift threshold. The repeated entries are useful evidence, but they should not be counted as unique attack events without further verification.

Scenario 2: False Data Injection

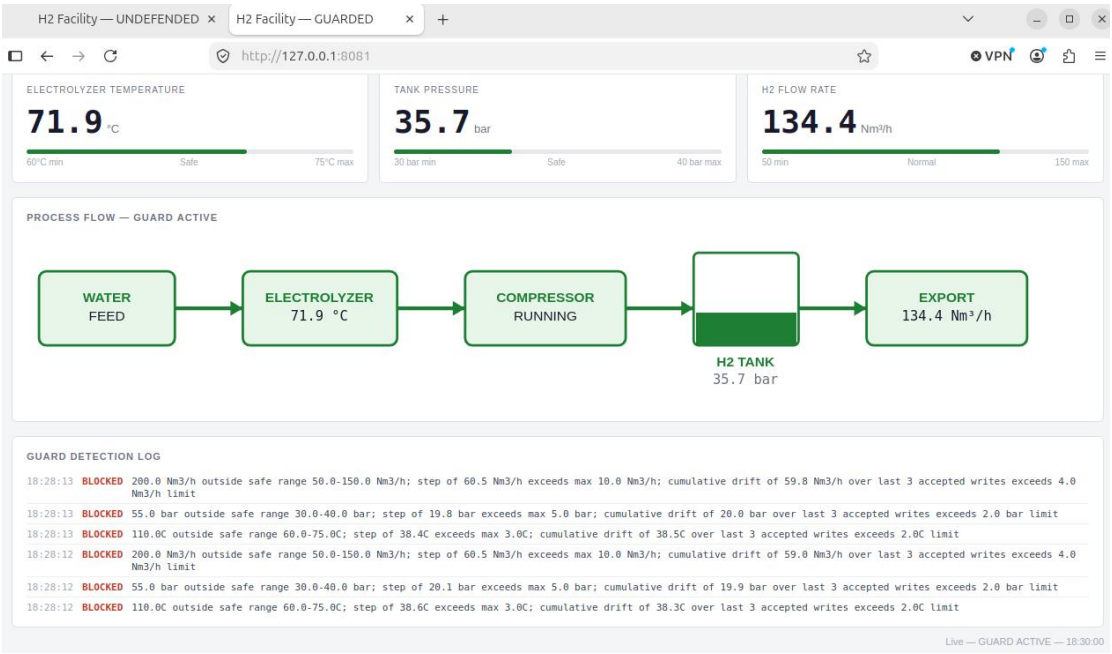
scenario2-direct-output



scenario2-direct-recovery



scenario2-Guarded-recovery



1. Direct Path - Port 5020

The command python3 false_data_attack.py 5020 produced the following results:

Process variable	Peak observed	Safe range	Unsafe readings
Temperature	110.0°C	60–75°C	6/6
Pressure	55.0 bar	30–40 bar	6/6
Flow rate	200.0 Nm³/h	50–150 Nm³/h	6/6

Observation: The direct path allowed the script to observe abnormal values for all three process variables across all six batches. The script reported UNSAFE VALUES REACHED THE PLANT, indicating that the tested writes were not prevented from producing out-of-range readings.

Recovery observation: The direct dashboard subsequently displayed 72.1°C, 36.5 bar, and 123.6 Nm³/h. All three values were within their configured ranges, and the dashboard displayed SYSTEM NORMAL.

2. Guarded Path - Port 5021

The command python3 false_data_attack.py 5021 produced the following results:

Process variable	Peak observed	Safe range	Unsafe readings
Temperature	71.7°C	60–75°C	0/6
Pressure	35.2 bar	30–40 bar	0/6
Flow rate	142.1 Nm³/h	50–150 Nm³/h	0/6

Observation: The guarded path reported no out-of-range readings across the six batches and returned attack fully contained. The guard detection log recorded BLOCKED attempts involving 110.0°C, 55.0 bar, and 200.0 Nm³/h. The log identified violations of the configured range, write-step, and cumulative-drift limits.

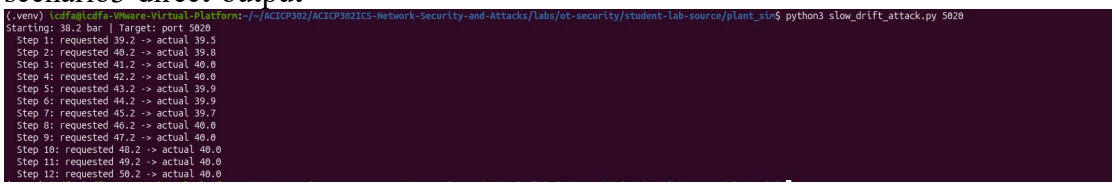
Recovery observation: The guarded dashboard subsequently displayed 72.7°C, 37.0 bar, and 141.5 Nm³/h. All values were within the configured ranges, and the dashboard displayed PROTECTED.

Conclusion

The Scenario 2 results show a clear difference between the two paths. The direct path reported unsafe readings for temperature, pressure, and flow in all six batches, whereas the guarded path reported zero unsafe readings and logged blocked attempts. Subsequent dashboard readings for both paths were within the configured ranges.

These findings support the effectiveness of the guard against this specific false-data injection demonstration, but do not establish that it can prevent every possible attack.

scenario3-direct-output

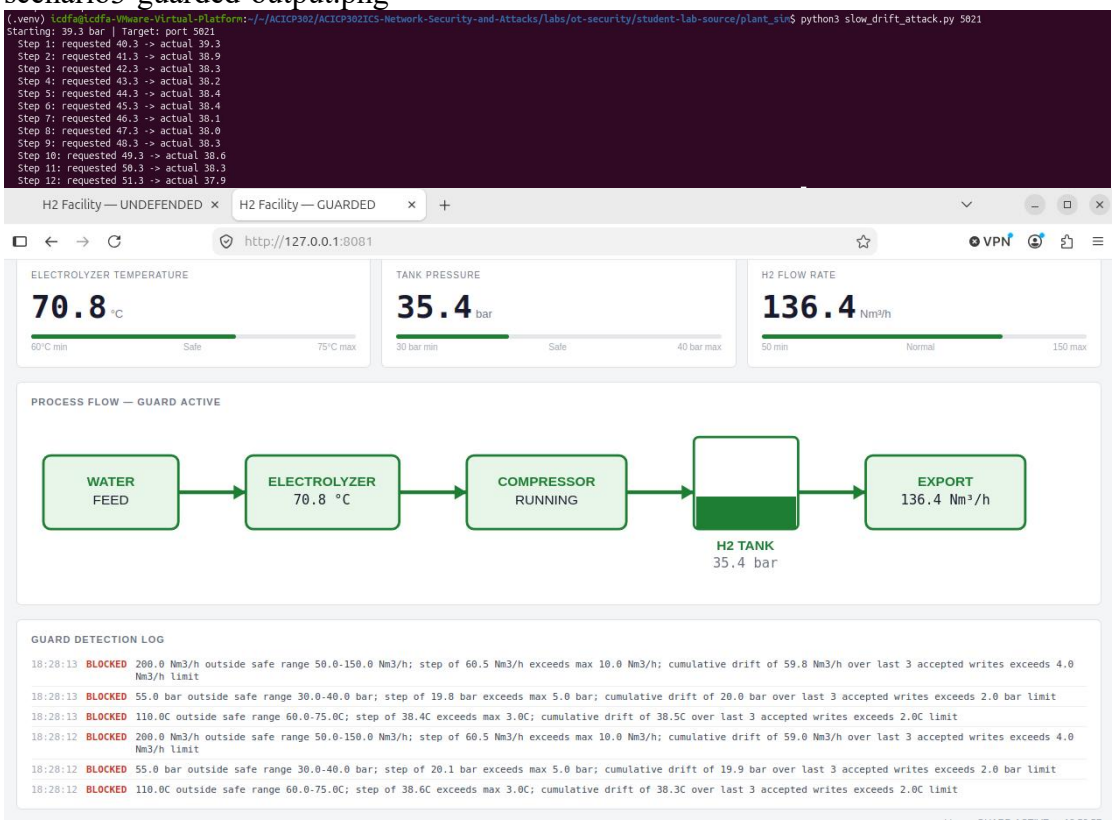


Results

Measurement	Direct path (5020)	Guarded path (5021)
Temperature	73.8°C	74.8°C
Tank pressure	40.0 bar	39.5 bar
H_2 flow rate	143.7 Nm^3/h	144.5 Nm^3/h
Displayed status	System Normal	Protected
Guard active	No	Yes

The configured limits are 60–75°C, 30–40 bar, and 50–150 Nm^3/h. All six dashboard readings are within their displayed ranges. However, the direct pressure is exactly at the upper limit, and both temperatures are approaching the upper limit.

scenario3-guarded-output.png



Results recorded

Measurement	Direct path (5020)	Guarded path (5021)
Starting pressure	38.2 bar	39.3 bar
Requested pressure range	39.2–50.2 bar	40.3–51.3 bar

Actual

Preliminary comparison

Observation	Direct (5020)	Guarded (5021)
Requested pressure range	39.2–50.2 bar	40.3–51.3 bar
Reported actual pressure	39.5–40.0 bar	37.9–39.3 bar
Highest reported actual pressure	40.0 bar	39.3 bar
Above 40-bar limit in reported readings	No	No

Interpretation: Neither script output shows a reported actual pressure above 40 bar. The guarded path maintained a lower maximum pressure in this run. We still need the post-test dashboard and current log evidence before finalizing the result and recovery assessment.

Activity 07 - Recovery, evidence and project defence

